

Sound State Encodings in Translational Separation Logic Verifiers (Extended Version)

HONGYI LING, ETH Zurich, Switzerland

THIBAUT DARDINIER, EPFL, Switzerland

ELLEN ARLT, MPI-SWS, Germany

PETER MÜLLER, ETH Zurich, Switzerland

Automated program verifiers are often organized into a front-end, which encodes an input program into an intermediate verification language (IVL), and a back-end, which proves that the IVL program is correct. Soundness of such translational verifiers requires that the back-end verification is sound and that correctness of the IVL program implies correctness of the input program. Existing formalizations for translational verifiers based on separation logic target the former, but support the latter only under the strong assumption that there exists a separation logic for the input program with the same state model as the IVL. This assumption is unrealistic in practice, especially since the state model also defines the supported separation logic resources.

We present the first formal framework for proving the soundness of translational separation logic verifiers with non-trivial state encodings. To be applicable to various front-ends and IVLs, our framework only assumes the existence of a homomorphic encoding relation between the front-end and IVL state models. At the core of our framework is a novel condition, backward satisfiability, which is crucial to guarantee the soundness of the front-end translation. We formalize our framework for front-end verifiers based on concurrent separation logic and separation logic IVLs, such as Raven, VeriFast, and Viper. We demonstrate its expressiveness by proving soundness for three common state encodings. Our framework and all proofs are formalized in Isabelle/HOL.

CCS Concepts: • **Theory of computation** → **Logic and verification**; **Automated reasoning**; **Separation logic**; **Axiomatic semantics**; **Program verification**.

Additional Key Words and Phrases: Translational Verifier, Front-End Translation, Intermediate Verification Language, Implicit Dynamic Frames, State Encoding, Backward Satisfiability

1 Introduction

Many modern program verification tools are *translational verifiers*, i.e., they split the verification into *front-ends*, which translate the input program with its specification into an *intermediate verification language* (IVL), and *back-ends*, which verify the resulting IVL program. This split enables the reuse of back-end verifiers across multiple front-ends, thus dramatically reducing the effort of developing verifiers. Examples of translational verifiers include Civi [25] and Dafny [28] based on the Boogie IVL [27], Gillian for C and Javascript [33] and Rust [2] based on GIL [17], Prusti [1], Nagini [14], and Gobra [58] based on Viper [34], as well as Frama-C [8] and Creusot [12] based on WhyML [15]. To support heap-manipulating and concurrent programs, many existing translational verifiers and their IVLs (GIL, Raven [19], VeriFast¹ [22], and Viper) are based on separation logic (SL) [42].

Soundness of a translational verifier requires (1) soundness of the back-end verifier and (2) soundness of the front-end translation, that is, that the successful verification of the IVL program implies correctness of the input program. While several existing works target back-end soundness for various IVLs [32, 37, 38, 7, 20], the state of the art does not offer practical techniques to prove soundness of realistic front-ends based on SL.

¹VeriFast does not have a human-readable IVL, but uses a uniform internal representation and back-end for multiple input languages (C, Java, Rust).

```

def main(v: Int):
  x := alloc(f)
  // { true }      { x.f ↦ _ }
  y := v || x.f := v
  // { y = v }      { ∃v'. x.f ↦ v' * v' = v }
  assert y = x.f

```

(a) Python-like front-end program, with concurrency and lazy field creation. The comments before and after the parallel composition represent the pre- and postconditions of the two threads, respectively. Here $x.f \mapsto _$ represents the ownership of $x.f$ *without* any guarantee that the field f has been created yet, while $\exists v'. x.f \mapsto v'$ represents the ownership of $x.f$ *with* the guarantee that it has been created.

<pre> method main(v: Int) { // x := alloc(f) havoc x inhale x.f_added ↦ false // parallel composition exhale true exhale ∃b. x.f_added ↦ b * (b ⇒ x.f_IVL ↦ _) havoc y inhale y = v inhale ∃v'. x.f_added ↦ true * x.f_IVL ↦ v' * v' = v // assert y = x.f assert ∃v'. x.f_IVL ↦ v' * y = v' } </pre>	<pre> method leftThread(v, y: Int) { inhale true y := v exhale y = v } method rightThread(v: Int, x: Ref) { inhale ∃b. x.f_added ↦ b * (b ⇒ x.f_IVL ↦ _) // x.f := v if (¬x.f_added) inhale x.f_IVL ↦ _ x.f_added := true x.f_IVL := v exhale ∃v'. x.f_added ↦ true * x.f_IVL ↦ v' * v' = v } </pre>
---	---

(b) Example translation of the front-end program into the Viper IVL, inspired by Nagini. The highlighted assertions represent the translations of the corresponding pre- and postconditions from Fig. 1a.

Fig. 1. An example front-end translation.

1.1 Challenge: Large Semantic Gap

The main challenge in establishing the soundness of a front-end translation into an SL-based IVL is to bridge the large semantic gap between the semantics and proof rules of the front-end programming language and the IVL. As an example, consider the program in Fig. 1a. This program allocates a new object with field f , then concurrently assigns the value of the argument v to local variable y and field $x.f$, before asserting that y and $x.f$ are equal. Like in Python, the language used here adds the field f lazily to the object when it is first written: Attempting to read a field that has not yet been added results in a runtime error. To verify this program, a translation into the IVL must check that all fields that are read were previously assigned, in addition to ensuring the absence of memory errors and data races, and that the assertion at the end of method `main` holds.

Concurrency. Our IVL translation, inspired by the Viper-based Python verifier Nagini [14], is shown on Fig. 1b. Since the IVL does not support concurrency directly, we translate it using concurrent separation logic (CSL) [36] rules: The two front-end threads with their own pre- and postconditions are translated into and verified as separate IVL methods `leftThread` and `rightThread`.

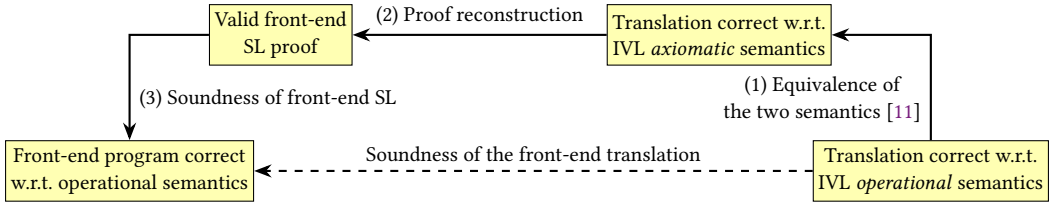


Fig. 2. Decomposing the soundness proof via a front-end SL. The dashed arrow represents the goal and the three solid arrows represent the decomposition. Existing work does the decomposition, but assumes that the two SLs have the same state model, which is unrealistic in practice. Our work lifts this key restriction, and enables full soundness proofs for front-ends with different state models to their IVLs.

In each of them, the translated precondition is added to the initially-empty IVL state via an **inhale** (also called *produce*) operation, the translated thread body is executed, and the translated postcondition is checked (and removed from the final state) via **exhale** (also called *consume*). In the main method, the concurrent part is abstracted to an exhale-havoc-inhale sequence that first removes the resources transferred to each thread via **exhales**, then reflects the side effects of the threads on the local variable y by assigning a non-deterministic value (**havoc**), and finally adds the resources given back by each thread via **inhales**.

Lazy field creation. As our target IVL has static field declarations (but does not support lazy field creation directly), our encoding needs to reflect for each field whether it has been added. To this end, the encoding represents each field f of the front-end program with two statically-declared fields in the IVL: the field f_{IVL} to store the value of the front-end field, and an additional boolean field f_{added} that indicates whether field f has been added to the object. As shown in the example: (1) Allocating the object x (as in the translation of $x := \text{alloc}(f)$) produces the SL resource $x.f_{added} \mapsto \text{false}$, which provides the permission to add the field later, but does *not* generate any ownership to f_{IVL} . (2) Assigning to $x.f$ (as in the translation of $x.f := v$ in the right thread) requires $x.f_{added} \mapsto \text{false}$ (if the field has not been added before) or $x.f_{IVL} \mapsto _$ (if it has). In the former case, the resource $x.f_{IVL} \mapsto _$ is produced lazily. In either case, $x.f_{IVL} \mapsto v$ is held after the assignment, thereby enabling subsequent reads. (3) Reading $x.f$ (as in the translation of **assert** $y = x.f$) requires $x.f_{IVL} \mapsto _$, as usual.

This example demonstrates that the gap between front-end programs and their IVL encoding is substantial, affecting the state representation (here, an extra IVL field per front-end field), the operational execution (e.g., concurrency in the front-end program vs. multiple sequential methods on the IVL), and proof rules (e.g., CSL for the front-end vs. **inhale** and **exhale** primitives in the IVL). A soundness proof for a front-end translation needs to bridge this gap to show that verification of the IVL program implies the correctness of the front-end program.

1.2 State of the Art

As explained above, establishing the soundness of a front-end translation into an SL-based IVL, for example for the translation depicted in Fig. 1, requires bridging the large semantic gap between the semantics and proof rules of the front-end programming language and the IVL. Dardinier et al. [11] show that this gap can be reduced to proving that the correctness of the IVL encoding (with respect to an *axiomatic* semantics) implies the existence of a valid proof in a suitable *program logic* for the front-end programming language, as depicted in Fig. 2. This decomposition leverages the existence of sound separation logics for many advanced front-end features (e.g., different kinds of

concurrency [36, 18, 13, 56, 24], initialization semantics [41], non-local control flow [26], etc.) to prove the soundness of front-end translations. For example, Dardinier et al. show how to prove the soundness of the concurrency translation from Fig. 1. However, their framework requires the front-end SL and the IVL to use the same state model, which is unrealistic in practice.

In practice, most front-ends use a state model that is vastly different from the fixed state model of the IVL. One major discrepancy is how memory and data are represented. For example, C’s memory regions or Java’s arrays might be encoded into an object-based memory model in an IVL, which does not natively support a notion of consecutive addresses. Moreover, our example from Fig. 1a encodes each Python field by a pair of fields to track which fields have been added to an object.

Important discrepancies also arise for the information tracked as part of the verification state, such as the separation logic resources held in a state. For example, front-end states may include designated resources to execute a method call [21] or perform an I/O operation [39, 45], which need to be encoded into standard SL predicates in the IVL. Moreover, front-end and IVL may use a different permission model. For example, Viper [34], VeriFast [22], and Raven² [19] all build in fractional permissions [6, 5] but have front-ends that use binary³ permissions (e.g., the encoding of I/O tokens in VeriFast [39] or Viper [45, 40]), counting permissions [5] (e.g., in Gobra [58] and the encoding of relaxed separation logic [56] in Viper [46, 34]), or duplicable permissions (e.g., duplicable invariants over immutable shared data [41]).

Due to these discrepancies, none of these encodings can be proven sound by the existing framework by Dardinier et al.

1.3 This Work

We develop the first formal framework for proving the soundness of front-end translations that use different state models from their IVLs, thus lifting the key restriction of existing work and enabling soundness proofs for practical front-end translations. To be compatible with a wide range of front-end separation logics and IVLs, our framework simply assumes the existence of a homomorphic encoding relation between front-end and IVL states. We identify a novel condition on this encoding relation, which we call *backward satisfiability*, that is crucial to guarantee the soundness of the front-end translation. To show the practicality of our approach, we develop a formal framework to prove the soundness of front-end translations based on concurrent separation logic (CSL) [36], and instantiate it with three different front-end translations inspired by existing Viper front-ends, including the one in Fig. 1. We also show that backward satisfiability is essential for soundness, by uncovering several subtle sources of unsoundness in state encodings that affect existing verifiers, but have not been described before. More broadly, we show that violations of our framework’s requirements underlie several previously-reported soundness bugs in verifiers such as Prusti, Nagini, and VerCors [4].

In addition to proving the soundness of existing front-ends, our framework also guides the development of new sound front-end translations. Developing a front-end typically involves three steps: choosing a suitable IVL, designing the state encoding relation between front-end and IVL states, and defining the translations of front-end expressions, assertions, and statements. Our framework’s requirements support each of these steps in two ways. First, they provide design guidance even before any proof is attempted: because a sound design must satisfy them, they rule out large families of unsound choices and narrow the design space. Second, they enable incremental

²Raven employs user-definable resource algebras for its ghost resources, but still uses fractional permissions for its regular program states.

³That is, one either owns a full permission or no permission at all.

verification: once a design decision is made, the corresponding requirement is a self-contained proof obligation that can be discharged immediately, allowing to catch an unsound choice early rather than after the whole translation is built.

In summary, we make the following technical contributions:

- (1) We identify a set of sufficient conditions, including the novel *backward satisfiability*, that guarantee the soundness of state encodings (§2 and §3).
- (2) We identify several subtle sources of unsoundness in state encodings that affect existing verifiers, but have not been described before (§2).
- (3) We develop a formal framework for proving the soundness of front-end translations. It is parametric in the front-end state model, expressions, assertions, statements, and logic, as well as their translations into the IVL. It is applicable to the translations used in practical SL-verifiers for both sequential and concurrent programs (§3).
- (4) We demonstrate the expressiveness of our framework by instantiating it for three practical front-end translations inspired by existing verifiers (§4).
- (5) The framework including all proofs is formalized in Isabelle/HOL.

Our framework, including all formalizations and proofs, is mechanized in Isabelle/HOL [35] and the mechanization is publicly available [31].

2 Overview: Homomorphism, Spurious Splittability, and Backward Satisfiability

In this section, we present key properties that a front-end translation must satisfy to be sound. We assume the existence of a state encoding relation between front-end and IVL states, which is general enough to capture many practical front-end translations (§2.1). We explain why this relation should be homomorphic, and show how violating this requirement can lead to unsoundness (§2.2). We then describe the key problem of *spurious splittability*, where the IVL state resulting from encoding a front-end state is split into two IVL states that have no front-end counterparts (§2.3). We explain how spurious splittability may lead to unsoundness and introduce our solution, *backward satisfiability*, to prevent this (§2.4). Finally, we show examples where spurious splittability, and the absence of backward satisfiability, lead to unsoundness (§2.5).

2.1 State Encoding Relations

Discrepancies between front-end and IVL states can stem from many different sources: a different heap structure (e.g., in Fig. 1), a different permission model (e.g., binary vs. fractional permissions), or even a different representation of the same permission model (e.g., rational vs. real numbers for fractional permissions). To abstract over these different aspects and make our approach generically applicable to different front-ends and IVLs, our framework is parametric in the front-end and IVL state models, and we simply assume the existence of a state encoding relation $\cdot \sim_S \cdot$ between them. Intuitively, $\omega \sim_S \Omega$ holds when the IVL state Ω represents the front-end state ω .

Relations from encoding functions. The state encoding *relation* is often derived naturally from a *function* $\llbracket \cdot \rrbracket_S$ that encodes a front-end separation logic state into the corresponding IVL state, in which case $\omega \sim_S \Omega$ is defined as $\Omega = \llbracket \omega \rrbracket_S$. For example, $\llbracket \cdot \rrbracket_S$ could encode binary permissions into fractional permissions by setting a permission of 1 for locations owned by the state, and 0 for the others. In the example from Fig. 1, our function $\llbracket \cdot \rrbracket_S$ encodes states as follows: The ownership of a non-existent field f of a created object x is encoded as the ownership of f_{added} of x with its value set to **false**; the ownership of a created field f with value v is encoded as the ownership of both f_{added} and f_{IVL} , where the former has value **true** and the latter stores the actual value v . The situation is represented in black in Fig. 3: Our function $\llbracket \cdot \rrbracket_S$ encodes the front-end states into

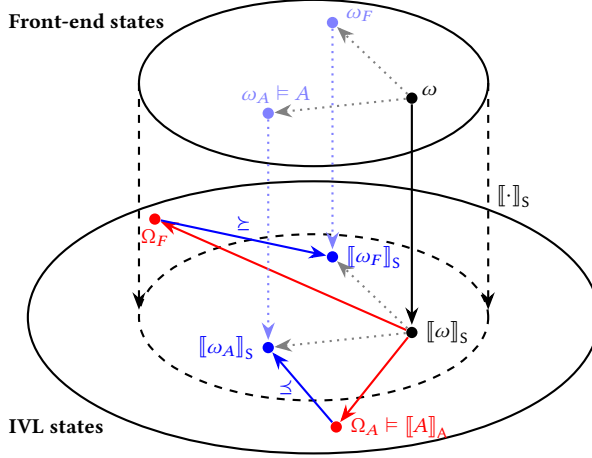


Fig. 3. An illustration of our setting (in black), the problem of spurious splittability (in red), and the backward satisfiability solution (in blue). The black dashed arrows represent the state encoding function $\llbracket \cdot \rrbracket_S$ from front-end states to IVL states, and the black dashed ellipse represents the image of $\llbracket \cdot \rrbracket_S$. The red arrows represent a possible spurious split of the IVL state $\llbracket \omega \rrbracket_S$ into Ω_A (that satisfies the translated assertion $\llbracket A \rrbracket_A$) and Ω_F (the frame), that have no front-end counterparts. The blue arrows show how the backward satisfiability of A guarantees the existence of front-end states ω_A and ω_F , such that ω_A satisfies A , $\llbracket \omega_A \rrbracket_S$ is smaller or equal to Ω_A , and $\llbracket \omega_F \rrbracket_S$ is larger or equal to Ω_F .

(typically) a subset of the IVL states. For example, the image of the encoding function from binary permissions is the subset of IVL states with permissions 0 or 1.

The need for a relation. State encoding functions are too restrictive to capture some front-end encodings, for instance, translations that encode duplicable permissions into fractional permissions, as no fixed positive fraction, however small, can be duplicated. However, duplicable permissions can be interpreted in the IVL as *any arbitrary* positive fraction: A front-end state with a single duplicable resource then corresponds to infinitely many IVL states, each with a different fraction. Such encodings are easily captured by our binary relation $\cdot \sim_S \cdot$, as we will show in §4.

2.2 The Homomorphism Requirement

At the core of separation logic is the concept of *separation*: A state ω satisfies the assertion $P * Q$ if it can be split into two states ω_P and ω_Q such that ω_P satisfies P , ω_Q satisfies Q , and $\omega = \omega_P \oplus \omega_Q$, where $\oplus$ is a partial commutative and associative binary operation. In particular, both front-end and IVL states are equipped with their operator $\oplus$. For the translation to be meaningful, we require the state encoding relation to be homomorphic with respect to $\oplus$.

For simplicity, we describe here the homomorphism requirement for the case where $\cdot \sim_S \cdot$ is defined by an encoding function $\llbracket \cdot \rrbracket_S$, but we adapt it later to the general case of a relation. The encoding function $\llbracket \cdot \rrbracket_S$ is required to be homomorphic: For all front-end states ω_1 and ω_2 , $\llbracket \omega_1 \oplus \omega_2 \rrbracket_S = \llbracket \omega_1 \rrbracket_S \oplus \llbracket \omega_2 \rrbracket_S$. This is necessary so that the separating conjunction ($*$) in the IVL program (e.g., when adding or removing resources via inhale or exhale) corresponds to the separating conjunction in the front-end program (e.g., in its parallel rule).

Unsoundness without homomorphism. To see why the homomorphism requirement is necessary, consider encoding a front-end state model with counting permissions into an IVL state model with fractional permissions. Counting permissions [5] is a permission model that allows one to

<pre> method m(x: Ref, y: Ref, z: Ref) requires x.f $\xrightarrow{\epsilon}$ - * y.f $\xrightarrow{\epsilon}$ - * z.f $\xrightarrow{\epsilon}$ - { if (x = y $\wedge$ x = z) assert false } </pre>	<pre> method m(x: Ref, y: Ref, z: Ref) { inhale x.f $\xrightarrow{\frac{1}{2}}$ - * y.f $\xrightarrow{\frac{1}{2}}$ - * z.f $\xrightarrow{\frac{1}{2}}$ - if (x = y $\wedge$ x = z) assert false } </pre>
--	---

Fig. 4. Encoding counting permissions $k \cdot \epsilon$ as fractions $1 - \frac{1}{2^k}$ is unsound. The front-end program on the left permits the three inputs to be aliased, but the IVL program on the right does not.

subtract from a full permission an unbounded number of units. In this model, a state holds either $1 - k \cdot \epsilon$ or $k \cdot \epsilon$ permission to a heap location (for any non-negative k), where a full permission 1 is required to write to a location, and any positive number of units $k \cdot \epsilon$ provides read access. Counting permissions are, for instance, used in the Chalice verifier [29].

A simple (but unsound) encoding of counting permissions into an IVL offering fractional permissions [6], such as VeriFast and Viper, is to encode $k \cdot \epsilon$ as the fraction $1 - \frac{1}{2^k}$, and $1 - k \cdot \epsilon$ as the complementary fraction $\frac{1}{2^k}$. This encoding ensures that the sum of the fractions for $1 - k \cdot \epsilon$ and $k \cdot \epsilon$ provides write permission and, for positive k , each fraction is positive and, thus, provides read permission. However, this encoding is *not* homomorphic: Let ω^k be a front-end state with $k \cdot \epsilon$ permission to some heap location $x.f$, and Ω^p be an IVL state with a fractional permission p to the corresponding heap location. Then $\llbracket \omega^1 \oplus \omega^1 \rrbracket_S = \llbracket \omega^2 \rrbracket_S = \Omega^{1 - \frac{1}{2^2}} = \Omega^{\frac{3}{4}}$, whereas $\llbracket \omega^1 \rrbracket_S \oplus \llbracket \omega^1 \rrbracket_S = \Omega^{1 - \frac{1}{2^1}} \oplus \Omega^{1 - \frac{1}{2^1}} = \Omega^1 \neq \Omega^{\frac{3}{4}}$. In other words, the encoding of the sum of the front-end states $\omega^1 \oplus \omega^1$ has *strictly less* permission than the sum of the individual encodings.

This violation of the homomorphic requirement results in an unsound front-end translation, as illustrated by the example in Fig. 4. The front-end program on the left might lead to an assertion violation because the counting permissions required in the precondition do *not* prevent all three inputs from being aliased, as there is no bound on the number of units for each location. In contrast, the assertion inhaled by the IVL program (on the right) rules out the case that all three inputs are aliased since it is not possible to own more than one full permission to each location. Therefore, the IVL program verifies even though the input program should not.

Note that this unsoundness is independent of the particular fraction chosen to represent one unit, as long as this fraction is strictly positive; smaller fractions just need more aliases to exhibit the unsoundness. Summers and Müller [46] use an alternative encoding, in which a unit is encoded as a *symbolic* fraction, together with the axioms $f > 0$ and $\forall k \in \mathbb{N}. k \cdot f < 1$. This representation makes it harder for an SMT-based verifier to *exploit* the unsoundness, but it is still present as there is no fraction f that satisfies these axioms.

2.3 Spurious Splittability

As discussed in §1, front-end translations rely on SL operations to add resources to the verification state (*produce*) and to remove them (*consume*). IVLs provide these operations (e.g., **inhale** and **exhale** in Viper) to encode SL proof rules. For example, the front-end translation from Fig. 1 uses these operations to model allocation and parallel composition, which are not supported by the IVL.

In general, a front-end statement C (e.g., $x := \text{alloc}(f)$) can be modeled in the IVL via the corresponding rule from the front-end separation logic: If the triple $\{P\} C \{Q\}$ is valid in the front-end separation logic, then a possible (and typical) translation for C in the IVL is **exhale** P ; **inhale** Q (assuming that C does not modify local variables for simplicity). Starting in the IVL state Ω , **exhale** P will split Ω into $\Omega_P \oplus \Omega_F$ (where Ω_P satisfies P) and remove Ω_P . The resulting state Ω_F

<pre> method main() { x := alloc(f) fork reader(x) y := x.f } method reader(x: Ref) requires x.f ↦ _ { var v := x.f ... } </pre>	<pre> method main() { havoc x inhale x.f ↦ _ exhale $\exists p > 0. x.f \xrightarrow{p} _$ y := x.f } method reader(x: Ref) { inhale $\exists p > 0. x.f \xrightarrow{p} _$ var v := x.f ... } </pre>
---	---

Fig. 5. Encoding binary permissions using fractional permissions is generally unsound. The IVL program on the right allows reading $x.f$ after the fork because of the use of fractional permissions), and thus is correct. In contrast, the front-end program uses binary permissions, and thus should not be verified.

represents the frame, *i.e.*, the resources that are unaffected by the execution of C . The **inhale** Q statement will then add another state Ω_Q to the frame Ω_F , resulting in the state $\Omega_Q \oplus \Omega_F$. The state Ω_Q , which satisfies Q , models how the statement C modified Ω_P . Intuitively, this encoding is justified by the frame rule of the front-end separation logic, with Ω_F used as frame.

Unfortunately, this intuitive justification might not be valid, because the split $\Omega_P \oplus \Omega_F$ might be *spurious*: As represented in Fig. 3 (in red), the IVL states Ω_P and Ω_F might not have front-end counterparts, even if the initial IVL state Ω has a front-end counterpart (*i.e.*, a front-end state ω such that $\Omega = \llbracket \omega \rrbracket_S$). In such a case, the frame rule of the front-end separation logic cannot be used soundly with the IVL state Ω_F , as it has only been proven sound for front-end states. Such spurious splits arise because the front-end states generally correspond to a *subset* of the IVL states; in other words, there are generally infinitely many more IVL states than front-end states, and thus infinitely many IVL states without front-end counterparts. We call IVL states that have a front-end counterpart *concrete* states, and *spurious* those that do not.

Unsoundness from spurious splits. As a simplified example (we show more examples below), consider a front-end that uses binary permissions. The encoding into an IVL with fractional permissions attempts to improve framing by determining with a simple pre-analysis whether a method updates a heap location. If not, it encodes permission to that location as an arbitrary positive fraction rather than full permission, such that callers can frame the value around calls. In this setting, the front-end program on the left of Fig. 5 does not verify because there is no permission to access $x.f$ after the fork. However, the encoding on the right verifies because the assertion $\exists p > 0. x.f \xrightarrow{p} _$ in the IVL program (on the right of Fig. 5) can be satisfied by any positive fraction, enabling the read $y := x.f$ in the main thread. This unsoundness is caused by spurious splittability: an IVL state with full permission to $x.f$ can be split into two states, each with $1/2$ -permission (one being passed to the forked thread and the other retained to read $x.f$ subsequently). However, these states have no corresponding front-end states, such that the same reasoning steps are not valid there. We show in §2.5 that similar problems arise even with an assertion syntax suitably restricted for the front-end state model.

2.4 Backward Satisfiability

The unsoundness in the previous example comes from the fact that the encoded precondition $\exists p > 0. x.f \xrightarrow{p} _$ can be satisfied by a spurious IVL state, which results in a spurious frame. In particular, no smaller concrete state satisfies the encoded precondition, such that *any* split that enables verifying the whole IVL program is spurious.

We prevent this source of unsoundness by requiring front-end assertions to satisfy a novel soundness condition. Intuitively, a front-end assertion A is *backward satisfiable* if for any⁴ IVL state Ω_A that satisfies its translation $\llbracket A \rrbracket_A$, there exists a front-end state ω_A that satisfies A and whose encoding $\llbracket \omega_A \rrbracket_S$ is smaller than Ω_A . Intuitively, and as depicted in Fig. 3 in blue, backward satisfiability guarantees the existence of an alternative, non-spurious split $\llbracket \omega \rrbracket_S = \llbracket \omega_A \rrbracket_S \oplus \llbracket \omega_F \rrbracket_S$ for some front-end states ω_A (satisfying A) and ω_F , such that $\llbracket \omega_F \rrbracket_S \succeq \Omega_F$. The latter ensures that the IVL frame Ω_F can be used soundly with the frame rule from the front-end SL.⁵

Importantly, backward satisfiability holds for many basic SL assertions, such as points-to $x.f \xrightarrow{p} v$ and pure assertions (*i.e.*, without permissions). Backward satisfiability is preserved by many important SL connectives. For example, if A and B are backward satisfiable, then so is $A * B$ (with the straightforward definition $\llbracket A * B \rrbracket_A \triangleq \llbracket A \rrbracket_A * \llbracket B \rrbracket_A$). Unfortunately, backward satisfiability is *not* preserved by all connectives, such as magic wands (separating implications), as we show next.

2.5 Unsoundness without Backward Satisfiability

In this subsection, we show two examples that violate backward satisfiability because of the magic wand connective, and how the soundness of front-end translations can be compromised when the backward satisfiability condition is violated. The framework presented in this paper allows one to uncover such subtle sources of unsoundness and to prove that correct encodings are indeed sound.

Encoding binary permissions into fractional permissions. The example in Fig. 6 encodes the input program on the left into the identical IVL program on the right. The front-end program on the left uses binary permissions, whereas the IVL encoding on the right uses fractional permissions. The assertion W is defined using a magic wand (separating implication) connective:

$$W \triangleq x.f \mapsto _ \multimap (x.f \mapsto _ * y.g \mapsto _)$$

Intuitively, it expresses that one can give up the wand as well as ownership of $x.f$ to obtain ownership of both $x.f$ and $y.g$ (that is, apply modus ponens).

<pre style="margin: 0;">method m(x: Ref) requires x.f ↦ _ ensures W * W * W { skip }</pre>	<pre style="margin: 0;">method m(x: Ref) { inhale x.f ↦ _ skip exhale W * W * W }</pre>
--	---

Fig. 6. Encoding binary permissions using fractional permissions is generally unsound. The IVL program on the right verifies because there are fractional permissions for the footprints of the three magic wands W . With the binary permissions used by the front-end program on the left, it is not possible to provide sufficient permissions for all three wands.

Magic wands themselves represent resources (the wand’s *footprint*) that, combined with the left-hand side of the wand, entail the right-hand side. With binary permissions, W ’s footprint

⁴More precisely, backward satisfiability only considers IVL states Ω_A that are smaller than the encoding $\llbracket \omega \rrbracket_S$ of some front-end state ω , *i.e.*, such that $\llbracket \omega \rrbracket_S = \Omega_A \oplus \Omega_R$ for some IVL state Ω_R .

⁵Throughout the paper, we assume an affine front-end SL.

contains either permission to $x.f$ (so that combining it with the left-hand side yields false, thereby trivially satisfying the right-hand side) or permission to $y.g$. No matter which footprint is chosen, the postcondition represents more than full permission to a heap location and is thus unsatisfiable. Hence, verification should fail. Nevertheless, verification of the IVL program succeeds. With fractional permissions, one can choose W 's footprint as $x.f \mapsto _$, such that the right-hand side is entailed trivially. With this choice, the postcondition holds, and verification succeeds.

This unsoundness stems from the violation of backward satisfiability by W (despite the fact that both sides of the magic wand are backward satisfiable): the IVL may satisfy the translation of W in a state Ω_A with $\frac{1}{3}$ permission to $x.f$. However, the *only* front-end state ω_A that satisfies W has full permission to $x.f$ and, thus, its encoding is *not* smaller than or equal to Ω_A .

This example also demonstrates that a commonly-used state encoding is sound only if it is combined with appropriate restrictions on the allowed assertions. For instance, disallowing binary permissions inside magic wands would rule out our unsound example.

Encoding rational permission amounts into real permission amounts. Our next example shows that the common practice of encoding SLs with rational permission amounts into automatic verifiers targeting real permission amounts is surprisingly unsound in the presence of the magic wand connective. Consider the following two assertions:

$$P_1 \triangleq \left(\exists v. y.g \mapsto v * 0 < v * v^2 \leq \frac{1}{2} \right) * \left(\exists v. y.g \mapsto v * x.f \mapsto _ \right)$$

$$P_2 \triangleq \left(\exists v. y.g \mapsto v * 0 < v < 1 * (1 - v)^2 \geq \frac{1}{2} \right) * \left(\exists v. y.g \mapsto v * x.f \mapsto _ \right)$$

In both of them, the left-hand side of the magic wand constrains the value v , which is used as a permission amount for $x.f$ on the right-hand side. The predicate $y.g \mapsto v$ enforces the values v on both sides to be the same. P_1 constrains v to be in the half-open interval $(0, \frac{1}{\sqrt{2}}]$, whereas P_2 allows values for v in $(0, 1 - \frac{1}{\sqrt{2}}]$. In both wands, the left-hand side provides no permission for $x.f$, so in both cases, $x.f \mapsto _$ is part of the footprint of the wand.

<pre>method m(x: Ref) requires x.f ↦ _ ensures P₁ * P₂ { skip }</pre>	<pre>method m(x: Ref) { inhale x.f ↦ _ skip exhale P₁ * P₂ }</pre>
---	--

Fig. 7. Encoding rational permission amounts as real numbers is unsound. The front-end program on the left and the IVL program on the right interpret the permission amounts in the postcondition differently, such that the IVL program verifies successfully even though the front-end program should not.

The front-end and IVL methods in Fig. 7 require full ownership to $x.f$ and ensure the separating conjunction $P_1 * P_2$. The two methods look identical, but use rationals and reals as permission amounts, respectively. The footprints of both magic wands need to contain $x.f \mapsto _$ for every value v permitted by the left-hand side. The IVL program, which uses real numbers as permission amounts, thus, uses $\frac{1}{\sqrt{2}}$ for P_1 and $1 - \frac{1}{\sqrt{2}}$ for P_2 . In real arithmetic, these add up to a full permission, which is provided by the precondition. Hence, verification of the IVL program succeeds. However, the front-end logic, which uses rational permission amounts, cannot represent the amounts $\frac{1}{\sqrt{2}}$ and $1 - \frac{1}{\sqrt{2}}$ precisely and, thus, needs to conservatively over-approximate them. As a result, their sum exceeds the full permission provided in the precondition, which causes verification to fail.

The unsoundness in this example again stems from the violation of backward satisfiability, by both P_1 and P_2 (despite the fact that all sides of the magic wands are backward satisfiable): the IVL may satisfy P_1 (resp. P_2) in a state Ω_A with $\frac{1}{\sqrt{2}}$ (resp. $1 - \frac{1}{\sqrt{2}}$) permission to $x.f$, while the front-end must provide strictly more amount of permission for ω_A to satisfy P_1 (resp. P_2).

This example shows that encoding rational permission amounts as reals is generally unsound. Since it should in principle be possible to formalize fractional permissions with real numbers instead of rational ones, we do not expect this unsoundness to compromise actual verification results. However, the mismatch makes it impossible to prove a formal soundness result for front-ends using this common state model encoding.

These unsoundnesses stem from the mismatch between the front-end and IVL state models: when the two coincide, the state encoding can be taken to be the identity, backward satisfiability holds trivially, and *all* connectives (including magic wands and universal quantifications) can be translated soundly. Conversely, for front-ends with a sufficiently expressive assertion language, backward satisfiability is necessary: whenever an assertion's translation violates it *and* the front-end can express the corresponding upper bound and frame (as in all our examples), one can construct an unsound method as above, in which the IVL translation verifies but the front-end program does not.

3 A Framework for Proving Front-end Translations Sound

Our core contribution is a formal framework that allows one to prove that the state model encoding of a front-end translation is sound. In this section, we formalize our framework: the semantic model of the IVL (§3.1), its parameters (§3.2) and their soundness requirements (§3.3), our soundness result (§3.4), and a generalization in which the state encoding is a relation rather than a function (§3.5). All results are formalized in Isabelle/HOL. Although the formalization uses Viper as the IVL, all requirements and results are independent of any specific IVL, so we present them for a general SL-based IVL.

3.1 Preliminaries

We adopt much of our semantic model of the IVL from Dardinier et al. [11].

IDF algebras. To make our framework widely applicable, our formalization captures both IVLs based on standard SL (e.g., VeriFast) and IVLs based on implicit dynamic frames (IDF) [43, 44] (e.g., Viper). To achieve that, we formalize states as IDF algebras, a generalization of separation algebras that also captures IDF state models:⁶

Definition 1 (IDF algebra). An IDF algebra is a tuple $(\Sigma, \oplus, |\cdot|, \text{stable}, \text{stabilize}, \text{unit})$ that satisfies a set of axioms (listed in Appendix A), where Σ is a set of states, $\oplus : \Sigma \times \Sigma \rightarrow \Sigma$ is a partial, commutative, and associative addition on Σ , $|\cdot|$, stabilize, and unit are endomorphisms of Σ , and stable is a predicate on Σ .

Intuitively, $\oplus$ denotes the composition of states, $|\cdot|$ projects a state on its largest duplicable part, stable decides the self-containedness of a state (i.e., the state holds enough ownership to all its fields so that none of them may be modified by the environment), stabilize projects a state on its largest stable part, and unit denotes the unit element of $\oplus$.

⁶Compared with the definition from Dardinier et al. [11], we make the unit function explicit and independent from stabilize and $|\cdot|$; the definition from Dardinier et al. [11] can be derived from ours by setting $\text{unit}(s) \triangleq \text{stabilize}(|s|)$. Our generalization is crucial to capture front-end state models where $\text{stabilize}(|\cdot|)$ does not satisfy the properties for $\text{unit}(\cdot)$, e.g., the immutable heaps from §4.4, where $\text{stabilize}(|h|) = h$ but $\text{unit}(h) = \lambda l. \perp$.

IDF assertions separate ownership information from value constraints. For instance, SL's points-to assertion $x.f \mapsto v$ is expressed in IDF as $\text{acc}(x.f) * x.f = v$. To ensure sound framing, IDF requires assertions to be self-framing; that is, an assertion may constrain a heap location only if it includes the permission to that location. Consequently, $\text{acc}(x.f) * x.f = v$ is self-framing, but $x.f = v$ alone is not. We defined self-framedness of assertions and other related concepts as follows:

Definition 2. Let P be an IDF assertion (i.e., a set of states from an IDF algebra).

- P is *self-framing*, written $\text{selfFraming}(P)$, iff $\forall \omega. \omega \in P \Leftrightarrow \text{stabilize}(\omega) \in P$.
- A state ω *frames* P , written $\text{frames}(\omega, P)$, iff $\text{selfFraming}(\{\omega\} * P)$.
- An IDF assertion B *frames* P , written $\text{frames}(B, P)$, iff $\forall \omega \in B. \text{stable}(\omega) \Rightarrow \text{frames}(\omega, P)$.
- P *frames an expression* (i.e., a partial function from states to values) e , written $\text{frames}(P, e)$, iff $e(\omega)$ is defined for all $\omega \in P$.
- An expression e is *stable under* P , written $\text{stableUnder}(e, P)$, iff $\forall \omega \in P. e(\omega)$ is defined iff $e(\text{stabilize}(\omega))$ is defined, and $e(\omega) = e(\text{stabilize}(\omega))$ when they are both defined.

IVL semantics. An IVL state consists of a store of local variables and a customized heap $\varphi \in \Sigma$ equipped with an IDF algebra structure, i.e., the set of IVL states $\Sigma_V = (\text{Var} \rightarrow V) \times \Sigma$, where V is the set of values and the store $\text{Var} \rightarrow V$ is instantiated to the agreement algebra, i.e., addition on stores is defined only for identical stores and yields the argument store, $|\cdot|$, stabilize, and unit are instantiated to the identity function, and $\text{stable}(s)$ holds for any store s .

We require that the IVL supports three primitive statements **havoc** x , **inhale** A , **exhale** A , and sequential compositions $C_1; C_2$ and conditional statements **if**(b) $\{C_1\}$ **else** $\{C_2\}$ in order to translate some front-end control structures. Intuitively, **havoc** x erases the value of local variable x by overwriting it non-deterministically, **inhale** A adds the resource specified by assertion A into the current state and assumes the value constraints in A , and **exhale** A checks that A holds in the current state and subsequently removes the resource corresponding to A from the current state.

We assume an axiomatic semantics for the IVL. The axiomatic semantic judgements are of the form $\Delta \vdash [P] C [Q]$, where Δ is a type context⁷, P and Q are IVL semantic assertions (i.e., sets of IVL states), and C is an IVL statement. The axiomatic semantic rules of the aforementioned statements are given in Appendix A.

3.2 Framework Parameters

To capture a wide range of front-ends, our framework offers numerous parameters that describe the input language to the front-end as well as the translation performed by the front-end.

Parameters describing the input language. The framework parameters for the input language describe, for instance, the available boolean expressions, assertions, and statements. Fig. 8 provides an overview of the parameters; in the following, we explain some of the most interesting ones; further details are included in the appendix.

Types and boolean expressions are fully described by the respective framework parameters (T_F and BE_F). The parameter A_{Fp} for assertions describes a set of so-called *input assertions*; the assertions supported by the front-end are then obtained using the following grammar, where $A_p \in A_{Fp}$, $b \in BE_F$, $t \in T_F$, and P is a designated symbol for recursive occurrences (as we explain below):

$$A ::= A_p \mid b \mid A * A \mid A \vee A \mid b \Rightarrow A \mid \exists x : t. A \mid P \mid \mu P. A$$

⁷We omit typing details here, but our Isabelle/HOL formalization includes them: it requires the state encoding to preserve well-typedness and deals only with well-typed states (whose store and heap contain values of the types prescribed by Δ). All states in §3 and §4 are implicitly assumed well-typed.

Framework parameter	Description
T_F	Set of front-end types.
V_F	Set of front-end values.
$\text{tp} : V_F \rightarrow T_F$	Function that assigns each value $v \in V_F$ a type $t \in T_F$.
H_F	Set of front-end <i>ghost heaps</i> used for reasoning about the resources in front-end states. They must be instantiated as an IDF algebra.
H_P	Set of front-end <i>program heaps</i> .
$\text{embed}_H : H_P \rightarrow H_F$	Function that embeds program heaps onto ghost heaps.
BE_F	Set of front-end boolean expressions.
$I_B : BE_F \rightarrow (\Sigma_F \rightarrow \{\text{true}, \text{false}\})$	Interpretation for each boolean expression $b \in BE_F$ as a partial predicate on front-end ghost states.
$\text{fv}_B : BE_F \rightarrow \mathcal{P}(\text{Var})$	(Possibly overapproximating) finite free variable set of a boolean expression.
A_{Fp}	Set of input assertions.
$\text{wf}_A^{\Delta_F}(\cdot)$	Well-formedness predicate on input assertions in a given type context Δ_F .
$I_A^{\Delta_F} : A_{Fp} \rightarrow \mathcal{P}(\Sigma_F)$	Interpretation of each input assertion $A_p \in A_{Fp}$ as a set of front-end ghost states satisfying the assertion under type context Δ_F .
$\text{fv}_A : A_{Fp} \rightarrow \mathcal{P}(\text{Var})$	(Possibly overapproximating) finite free variable set of an input assertion.
C_{Fp}	Set of primitive statements.
$\text{wf}_C^{\Delta_F}(\cdot)$	Well-formedness predicate on primitive statements in a given type context Δ_F .
Rules of the form $\sigma \xrightarrow{c}_{\Delta_F} \sigma'$	Operational semantic reduction rules for each primitive statement $c \in C_{Fp}$. $\sigma, \sigma' \in \Sigma_P$ are program pre- and post-states, and Δ_F is a type context.
Rules of the form $\Delta_F \vdash_P \{P\} c \{Q\}$	CSL rules for primitive statements $c \in C_{Fp}$. Δ_F is a front-end type context, and P and Q are front-end semantic assertions (<i>i.e.</i> , sets of front-end ghost states).
$\text{wr} : C_{Fp} \rightarrow \mathcal{P}(\text{Var})$	(Possibly overapproximating) finite modified variable set of a primitive statement.
$\text{fv}_C : C_{Fp} \rightarrow \mathcal{P}(\text{Var})$	(Possibly overapproximating) finite free variable set of a primitive statement.

Fig. 8. Framework parameters for the input language.

That is, separating conjunctions, disjunctions, implications, existential quantifiers, and inductive predicates are built into the framework, because they preserve backward satisfiability (as we present in Lemma 1) and we can thus reduce the effort of users of our framework to only proving backward satisfiability for input assertions; all other SL connectives (*e.g.*, magic wands) have to be provided as input assertions. Following Dardinier et al. [10], we formalize inductive predicates with one designated symbol P , which represents a recursive occurrence inside the least fixed-point computation $\mu P. A$. For example, the standard linked-list inductive predicate can be expressed as⁸

$$\mu P. x \neq \text{null} \Rightarrow (\text{acc}(x.\text{val}) * \text{acc}(x.\text{next}) * (\exists v : \text{Ref}. v = x.\text{next} * (\exists x : \text{Ref}. x = v * P)))$$

$I_A^{\Delta_F}$ is extended to an interpretation function on all front-end assertions using the standard interpretation for the built-in connectives and the least fixed point interpretation for inductive predicates. fv_A , the function that returns the set of free variables for assertions, and $\text{wf}_A^{\Delta_F}$, the well-formedness predicates of assertions, are extended to all front-end composite assertions accordingly. The concrete definitions of the extensions of $I_A^{\Delta_F}$, fv_A , and $\text{wf}_A^{\Delta_F}$ are in Appendix A.

Similarly, the parameter for statements C_p provides a set of *primitive* statements, from which the framework derives the set of statements using the following grammar, where $C_p \in C_{Fp}$, $b \in BE_F$, and A is a front-end assertion:

$$C ::= \text{skip} \mid C_p \mid C; C \mid \text{if}(b) \{C\} \text{ else } \{C\} \mid \text{while}(b) \text{ inv } A \{C\} \mid \{A\} C \{A\} \mid \{A\} C \{A\}$$

⁸The standard linked-list predicate is usually defined as $\text{List}(x) \triangleq x \neq \text{null} \Rightarrow (\text{acc}(x.\text{val}) * \text{acc}(x.\text{next}) * \text{List}(x.\text{next}))$. In our syntax, the recursive occurrence P (which in our example represents List) does not take explicit arguments as input; we thus use an existential quantifier to bind the heap-dependent expression $x.\text{next}$ to the implicit argument x .

Loops and parallel compositions contain assertions (invariants for loops, and pre- and postconditions for each parallel branch) for verification purposes. wf_C , fv_C , and wr can be extended to all front-end statements in a straightforward way, as shown in Appendix A.

The small-step operational semantic reduction rules and CSL rules for these control structures are fixed to the standard ones in the framework, as shown in Appendix A. For primitive statements, they are expressed by the framework parameters.

To customize the state model of the input language, the framework has parameters for types (T_F), values (V_F), program heaps (H_P), and ghost heaps (H_F). Intuitively, program heaps abstract the memory part of execution states and are involved in the definition of the operational semantics of the front-end language. In contrast, ghost heaps typically include the program heaps and additional SL resources (e.g., permission masks) for reasoning under SL. They are thus elements of an IDF algebra. We define front-end ghost states $\Sigma_F = (\text{Var} \rightarrow V_F) \times H_F$ and program states $\Sigma_P = (\text{Var} \rightarrow V_F) \times H_P$ to consist of a store of local variables and a ghost (resp. program) heap. The embedding function embed_H of heaps can then be naturally extended to an embedding $\text{embed}_S : \Sigma_P \rightarrow \Sigma_F$ of program states onto ghost states.

Parameters describing the front-end translation. Our framework captures the translation from the front-end language to the IVL by parameters $\llbracket \cdot \rrbracket_T$, $\llbracket \cdot \rrbracket_V$, $\llbracket \cdot \rrbracket_H$, $\llbracket \cdot \rrbracket_\Delta$, $\llbracket \cdot \rrbracket_B$, $\llbracket \cdot \rrbracket_{Ap}$, and $\llbracket \cdot \rrbracket_{Cp}$ for the translation of front-end types, values, ghost heaps, type contexts, boolean expressions, input assertions, and primitive statements, respectively. The functions $\llbracket \cdot \rrbracket_{Ap}$ and $\llbracket \cdot \rrbracket_{Cp}$ may translate front-end input assertions and primitive statements into composite ones in the IVL.

The translation of types and values may in general lose information. For example, when translating the front-end type **Rat** of rational values into the IVL type **Real** of real values, the rationality of front-end values will not be reflected in their IVL translation. As a result, witnesses for existential quantifiers in the IVL do not necessarily witness the corresponding quantifier in the input program, which could cause the translation to be unsound.

To avoid this unsoundness, our framework offers another parameter typeInv that is used to constrain the possible witnesses in the IVL-translation. For any front-end type $t \in T_F$ and local variable $x \in \text{Var}$, $\text{typeInv}(t, x)$ yields a syntactic boolean IVL expression that evaluates to **true** iff the value of variable x in the IVL state has a corresponding front-end value of type t . For example when $\llbracket \text{Rat} \rrbracket_T = \text{Real}$, we can define $\text{typeInv}(\text{Rat}, x) \triangleq (\exists a : \text{Int}. \exists b : \text{Int}. b \neq 0 \wedge x = a/b)$. Using this framework parameter, we extend the translation of input assertions to all front-end assertions and obtain the assertion translation function $\llbracket \cdot \rrbracket_A$. The definition of $\llbracket \exists x : t. A \rrbracket_A$ is as follows; that of the other cases is straightforward and is listed in Appendix A.

$$\llbracket \exists x : t. A \rrbracket_A \triangleq \exists x : \llbracket t \rrbracket_T. \text{typeInv}(t, x) * \llbracket A \rrbracket_A$$

Our framework translates loops and parallel compositions in an overapproximating way as verifiers like Viper do: the loop body or each thread in the parallel composition is translated together with its annotations as a separate program; we collect these programs to form a set of *auxiliary IVL statements* $\llbracket c \rrbracket_C^{\text{ext}}$ for the front-end statement c . If c does not contain loops or parallel compositions, $\llbracket c \rrbracket_C^{\text{ext}}$ is empty. In the main program, each statement c is translated to $\llbracket c \rrbracket_C^{\text{main}}$. If c is a loop or parallel composition, $\llbracket c \rrbracket_C^{\text{main}}$ is a sequence of exhale-havoc-inhale statements which first remove the resources specified by the annotations that are required to execute the block, then assign fresh values to all local variables that the execution of the block might modify, and finally add back the resources specified by the annotations that the execution of the block yields. Given $\llbracket \cdot \rrbracket_{Cp}$ that translates front-end primitive statements into (possibly composite) IVL statements, our framework thus defines the translation $\llbracket c \rrbracket_C$ of front-end composite statement c as a pair of an IVL statement $\llbracket c \rrbracket_C^{\text{main}}$ and a finite set of auxiliary IVL statements $\llbracket c \rrbracket_C^{\text{ext}}$. We give the definition of $\llbracket \cdot \rrbracket_C$

$$\begin{aligned}
\llbracket \text{while } (b) \text{ inv } I \{c\} \rrbracket_C &\triangleq \left(\text{exhale } \llbracket I \rrbracket_A * (\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B); \text{havoc } \text{wr}(c); \text{inhale } \llbracket I \rrbracket_A * \neg \llbracket b \rrbracket_B, \right. \\
&\quad \left. \left\{ \text{inhale } \llbracket I \rrbracket_A * \llbracket b \rrbracket_A; \llbracket c \rrbracket_C^{\text{main}}; \text{exhale } \llbracket I \rrbracket_A * (\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B) \right\} \cup \llbracket c \rrbracket_C^{\text{ext}} \right) \\
\llbracket \{P_1\} \ c_1 \ \{Q_1\} \ || \ \{P_2\} \ c_2 \ \{Q_2\} \rrbracket_C &\triangleq \left(\text{exhale } \llbracket P_1 * P_2 \rrbracket_A; \text{havoc } \text{wr}(c_1) \cup \text{wr}(c_2); \text{inhale } \llbracket Q_1 * Q_2 \rrbracket_A, \right. \\
&\quad \left\{ \text{inhale } \llbracket P_1 \rrbracket_A; \llbracket c_1 \rrbracket_C^{\text{main}}; \text{exhale } \llbracket Q_1 \rrbracket_A \right\} \cup \\
&\quad \left. \left\{ \text{inhale } \llbracket P_2 \rrbracket_A; \llbracket c_2 \rrbracket_C^{\text{main}}; \text{exhale } \llbracket Q_2 \rrbracket_A \right\} \cup \llbracket c_1 \rrbracket_C^{\text{ext}} \cup \llbracket c_2 \rrbracket_C^{\text{ext}} \right)
\end{aligned}$$

Fig. 9. The translation $\llbracket \cdot \rrbracket_C$ of front-end loops and parallel compositions. The full definition of $\llbracket \cdot \rrbracket_C$ is in Appendix A. The condition $\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B$ in the translation of loops excludes states in which the evaluation of $\llbracket b \rrbracket_B$ fails (e.g., due to a division by zero). Here we assume that the IVL supports (1) disjunction and negation of pure boolean expressions, and (2) directly using pure boolean expressions as assertions. It can therefore construct assertions including $\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B$. **havoc** V is a shorthand for **havoc** $x_1; \dots; \text{havoc } x_n$ for the finite set $V = \{x_1, \dots, x_n\}$.

for loops and parallel compositions in Fig. 9. That for the other control structures is straightforward; the full definition of $\llbracket \cdot \rrbracket_C$ is in Appendix A.

Finally, the provided parameters let us extend the encoding of front-end ghost heaps to an encoding $\llbracket \cdot \rrbracket_S$ of front-end ghost states: $\llbracket (s, \varphi) \rrbracket_S \triangleq (\llbracket s \rrbracket_{\text{St}}, \llbracket \varphi \rrbracket_H)$, where the store encoding $\llbracket s \rrbracket_{\text{St}}$ is defined as:

$$\llbracket s \rrbracket_{\text{St}}(x) = \begin{cases} \text{undefined}, & s(x) \text{ is undefined} \\ \llbracket s(x) \rrbracket_V, & \text{otherwise} \end{cases}$$

3.3 Requirements on the Framework Parameters

In this subsection, we present the requirements on the framework parameters that guarantee the soundness of the front-end translation. Most of these requirements for the parameters from Fig. 8 as well as the translations of types and values are standard and easy to satisfy; we list them in Appendix A.

Req. 1 states the homomorphism property of the state encoding. We state it on the ghost heap encoding parameter $\llbracket \cdot \rrbracket_H$ of the framework rather than the derived ghost state encoding $\llbracket \cdot \rrbracket_S$; the homomorphism result of $\llbracket \cdot \rrbracket_S$ can be easily derived from that of $\llbracket \cdot \rrbracket_H$ in Req. 1. The second property here is a sanity requirement that prevents the heap encoding from introducing garbage that has no correspondence in the front-end; while in the diagram of Fig. 3, backward satisfiability provides ω_A , this property guarantees the existence of ω_F and thus helps complete the desired state split in the front-end. Moreover, we require a third property, which allows us to support IDF.

Requirement 1. *The encoding of ghost heaps $\llbracket \cdot \rrbracket_H$ must satisfy the following properties:*

- $\llbracket \cdot \rrbracket_H$ preserves addition $\oplus$ when the sum of ghost heaps is defined in the front-end, i.e., if $\varphi_1 \oplus \varphi_2$ is defined, then so is $\llbracket \varphi_1 \rrbracket_H \oplus \llbracket \varphi_2 \rrbracket_H$ and $\llbracket \varphi_1 \rrbracket_H \oplus \llbracket \varphi_2 \rrbracket_H = \llbracket \varphi_1 \oplus \varphi_2 \rrbracket_H$.
- $\llbracket \cdot \rrbracket_H$ preserves subtraction in the following sense: if $\llbracket \varphi \rrbracket_H = \Phi_1 \oplus \llbracket \varphi_2 \rrbracket_H$, then there exists some φ_1 such that $\Phi_1 \succeq \llbracket \varphi_1 \rrbracket_H$ and $\varphi = \varphi_1 \oplus \varphi_2$, where $a \succeq b \triangleq (\exists r. a = b \oplus r)$.
- $\llbracket \cdot \rrbracket_H$ preserves stability. Specifically, $\text{stabilize}(\llbracket \varphi \rrbracket_H) = \llbracket \text{stabilize}(\varphi) \rrbracket_H$ for any front-end ghost heap $\varphi \in H_F$.

With the parameters of the front-end language and the translation from §3.2, we formally define backward satisfiability for front-end assertions in Def. 3.

Definition 3 (Backward satisfiability). A front-end assertion A is *backward satisfiable* w.r.t. the assertion translation $\llbracket \cdot \rrbracket_A$ iff: for any IVL state Ω that satisfies $\llbracket A \rrbracket_A$, and that is upper-bounded by

the encoding of some front-end state ω_b , i.e., $\llbracket \omega_b \rrbracket_S \succeq \Omega$, there exists a front-end state ω which satisfies A and $\Omega \oplus \llbracket \omega_b \rrbracket_S \succeq \llbracket \omega \rrbracket_S$ ⁹.

Req. 2 formalizes that the translations of boolean expressions, assertions, and statements must preserve the semantics in the front-end. For assertions, we require in addition monotonicity of translated assertions, a property that is naturally satisfied in an affine logic, and that the translation does not add new free occurrences of P . Together with backward satisfiability, it ensures that whenever the IVL satisfies a separating conjunction, the front-end can also satisfy it by redistributing the resources between the conjuncts. We make the requirements only for input assertions in **Req. 2**, and prove **Lemma 1** as part of the framework that these properties hold for all front-end assertions.

Requirement 2. *The translation functions for boolean expressions, input assertions, and primitive statements must satisfy the following properties:*

- The translation $\llbracket b \rrbracket_B$ of any front-end boolean expression $b \in BE_F$ preserves the semantics of b , i.e., for any front-end state $\omega \in \Sigma_F$, $I_B(b)(\omega)$ is defined iff $\llbracket b \rrbracket_B(\llbracket \omega \rrbracket_S)$ is defined, and they evaluate to the same truth value when both are defined.
- The translation of input assertions $\llbracket \cdot \rrbracket_{Ap}$ must satisfy:
 - $\llbracket \cdot \rrbracket_{Ap}$ preserves the semantics of all well-formed input assertions (i.e., each A such that $wf_A^{\Delta F}(A)$) in the front-end. Concretely, for any front-end state ω and any input assertion A , ω satisfies A iff $\llbracket \omega \rrbracket_S$ satisfies $\llbracket A \rrbracket_{Ap}$.
 - $\llbracket A \rrbracket_{Ap}$ is monotone for any input front-end assertion A . An assertion Q is monotone iff for any states Ω and Ω' , if Ω satisfies Q and $\Omega' \succeq \Omega$, then Ω' also satisfies Q .
 - $\llbracket A \rrbracket_{Ap}$ of any input front-end assertion A does not contain any free occurrence of P .
 - All well-formed input assertions are backward satisfiable w.r.t. $\llbracket \cdot \rrbracket_{Ap}$.
- The translation of statements $\llbracket \cdot \rrbracket_{Cp}$ preserves the semantics in the front-end, see formalization in **Appendix A**.

Lemma 1 (Extending requirements to all front-end assertions). *If the translation $\llbracket \cdot \rrbracket_A$ is monotone, does not add new free occurrences of P , and preserves the semantics for front-end assertions A_1 and A_2 , and these two assertions are backward satisfiable w.r.t. $\llbracket \cdot \rrbracket_A$, then these properties are preserved by $\llbracket \cdot \rrbracket_A$ of $A_1 * A_2$, $A_1 \vee A_2$, $A_1 \Rightarrow A_2$ (for a pure A_1), $\exists x : t. A_1$, and $\mu P. A_1$.*

As a corollary, $\llbracket \cdot \rrbracket_A$ is monotone and preserves the semantics for all well-formed front-end assertions, and all well-formed front-end assertions are backward satisfiable w.r.t. $\llbracket \cdot \rrbracket_A$.

In contrast, magic wands $*$, universal quantifications $\forall$, and iterated separating conjunctions $\otimes$ do not necessarily preserve backward satisfiability of their components. For instance, assertions W , P_1 , and P_2 from the two examples in §2.5 are not backward satisfiable (as explained in the corresponding examples), even though their components are by **Lemma 1**. For assertions containing these connectives, backward satisfiability needs to be proven without the help from **Lemma 1**.

3.4 Soundness

The conditions presented in the previous subsection are sufficient to guarantee that the front-end translation described by a given instantiation of our framework is sound. In the following, we summarize the soundness argument. The full proof is in our Isabelle/HOL formalization.

One of our central lemmas shows that the state encoding preserves the validity of separating conjunctions if the first conjunct is monotone and the second is backward satisfiable. This property

⁹The stronger statement $\Omega \succeq \llbracket \omega \rrbracket_S$ from **Fig. 3** might not be satisfied by some front-ends (e.g., the one encoding lazy field creation in §4.3) in the IDF algebra setting, as Ω might lose some pure information from ω_b which is necessary to construct ω .

is essential to show that assertions retain the meaning, no matter whether they are assumed or asserted. It is, in fact, one of the main reasons we require assertions to be backward satisfiable:

Lemma 2 (Translation of separating conjunctions). *For any IVL semantic assertion A , define $\text{btr}(A) \triangleq \{\omega \mid \llbracket \omega \rrbracket_S \in A\}$ as the backward translation of A , i.e., the collection of all front-end states whose encodings satisfy A .*

*For arbitrary IVL semantic assertions A and B , $\text{btr}(A) * \text{btr}(B) \subseteq \text{btr}(A * B)$. Furthermore, when A is monotone, and B is the translation of some backward satisfiable front-end assertion, the other direction also holds, i.e., $\text{btr}(A * B) \subseteq \text{btr}(A) * \text{btr}(B)$.*

Lemma 2 allows us to prove Lemma 3 which states that the exhale-havoc-inhale translation pattern that appears in the translation of both loops and parallel compositions (as in Fig. 9) is sound. The proof proceeds by applying the CSL frame rule on the CSL proof from the first property of Lemma 3 with a carefully chosen frame assertion, followed by an application of the consequence rule where the implications at the pre- and postconditions in the consequence rule are justified by the two directions of implication in Lemma 2 respectively.

Lemma 3 (Soundness of the exhale-havoc-inhale translation pattern). *Assume that the framework parameters satisfy Req. 1 and Req. 2, P_0 and Q_0 are well-formed, and that the following properties hold:*

- $\Delta_F \vdash_{\text{CSL}} \{I_A^{\Delta_F}(P_0)\} c \{I_A^{\Delta_F}(Q_0)\}$. That is, the front-end loop or parallel composition block has a proof under CSL against the pre- and postconditions from the annotation of the block.
- $\llbracket \Delta_F \rrbracket_\Delta \vdash [P] \text{ exhale } \llbracket P_0 \rrbracket_A; \text{ havoc } \text{wr}(c); \text{ inhale } \llbracket Q_0 \rrbracket_A [Q]$. That is, the main program of the translation of the loop or parallel composition verifies against another given pair of pre- and postconditions in the IVL.
- The given IVL pre- and postconditions P and Q are monotone.
- All written variables of the loop or parallel composition block are declared, and thus included in the type context, i.e., $\text{wr}(c) \subseteq \text{dom}(\Delta_F)$.

Then $\Delta_F \vdash_{\text{CSL}} \{\text{btr}(P)\} c \{\text{btr}(Q)\}$. That is, the front-end CSL proof for the block can be lifted to against the new pre- and postconditions backward translated from the IVL.

We can then prove our central soundness result stated as Thm. 4. The proof proceeds by induction on the structure of front-end statements, where for the loop and parallel composition cases, we can apply Lemma 3 with the IVL assertions P and Q in the lemma as the translation of the desired front-end pre- and postconditions $\llbracket P \rrbracket_A$ and $\llbracket Q \rrbracket_A$ and the first property derived by applying induction hypotheses on the axiomatic semantic proofs of the auxiliary IVL statements and CSL loop rule or parallel rule subsequently.

Theorem 4 (Soundness). *Assume that the framework parameters satisfy Req. 1 and Req. 2. Let c be a well-formed front-end composite statement w.r.t. type context Δ_F , and $\llbracket c \rrbracket_C = (C_m, C_e)$ be the translation of c . Let P and Q be arbitrary well-formed front-end composite assertions. Assume (1) $\llbracket \Delta_F \rrbracket_\Delta \vdash [\llbracket P \rrbracket_A] C_m [\llbracket Q \rrbracket_A]$, and (2) any $C \in C_e$ verifies in the IVL, i.e., there exists an IVL semantic assertion Q_C such that $\llbracket \Delta_F \rrbracket_\Delta \vdash [\top] C [Q_C]$. Then $\Delta_F \vdash_{\text{CSL}} \{I_A^{\Delta_F}(P)\} c \{I_A^{\Delta_F}(Q)\}$ holds in CSL.*

This theorem guarantees that, whenever the IVL translation of a front-end program is correct then there exists a proof in CSL for the front-end program. To further connect this result to the operational semantics of the front-end language, we also prove the soundness of the CSL rules w.r.t. the small-step semantics. Our proof adapts the proof from Vafeiadis [55] to our IDF setting. The formal statement of this result is in Appendix A.

3.5 Generalization to State Encoding Relations

As we have seen in §2.1, some front-ends might relate a single front-end state with multiple IVL states, as in the case of encoding duplicable permissions. To capture such encodings, we generalize the state encoding from a function $\llbracket \cdot \rrbracket_S$ to a binary relation $\cdot \sim_S \cdot$ between front-end states and IVL states and write $\omega \sim_S \Omega$ if IVL state Ω is *one of the possible encodings* of front-end state ω .

The generalized framework takes the same inputs as in Fig. 8, except that the heap encoding function $\llbracket \cdot \rrbracket_H$ is replaced by a binary relation $\cdot \sim_H \cdot$ that models the encoding of front-end ghost heaps into IVL heaps. The framework extends $\cdot \sim_H \cdot$ naturally to the binary relation $\cdot \sim_S \cdot$ between front-end and IVL states: $(s, h) \sim_S (S, H) \triangleq (S = \llbracket s \rrbracket_{St} \wedge h \sim_H H)$.

Most requirements of our framework are not affected by this generalization, but Req. 1 and Req. 2 need to be adapted:

Requirement 1'. *The binary relation $\cdot \sim_H \cdot$ between front-end ghost heaps and IVL heaps need to satisfy the following properties.*

- If $\varphi = \varphi_1 \oplus \varphi_2$ and $\varphi \sim_H \Phi$, then there exist Φ_1 and Φ_2 such that $\Phi = \Phi_1 \oplus \Phi_2$, $\varphi_1 \sim_H \Phi_1$, and $\varphi_2 \sim_H \Phi_2$.
- If $\varphi \sim_H \Phi$ and $\varphi_0 = \varphi \oplus \varphi_1$, then there exists Φ_1 such that $\varphi_1 \sim_H \Phi_1$, $\Phi \oplus \Phi_1$ is defined, and $\varphi_0 \sim_H (\Phi \oplus \Phi_1)$.
- For every front-end ghost heap $\varphi \in H_F$, there exists an IVL heap $\Phi \in \Sigma$ such that $\varphi \sim_H \Phi$.
- If $\varphi \sim_H \Phi$, then $\text{stabilize}(\varphi) \sim_H \text{stabilize}(\Phi)$.
- If $\text{stabilize}(\varphi) \sim_H \Phi'$, then $\exists \Phi$ such that $\text{stabilize}(\Phi) = \text{stabilize}(\Phi')$ and $\varphi \sim_H \Phi$.

Compared with Req. 1 in §3.3, the adaptation is in the following ways:

- The first property in Req. 1 (preservation of addition) is adjusted in a straightforward way.
- The second property in Req. 1 (preservation of subtraction) is now subsumed by a generalized backward satisfiability, see below.
- The second property here excludes certain ill-formed encodings by requiring that whenever a heap can be extended in the front-end, the corresponding extension is possible in the IVL (for instance, it is not possible to encode a fractional permission in the front-end to a full permission in the IVL). For the case where the heap encoding is a function, this property follows from the addition requirement and is, thus, not explicitly required in Req. 1.
- The third property here, intuitively, ensures that the encoding is non-degenerated.
- The preservation of stability (*i.e.*, the last requirement in Req. 1) is adapted to the last two requirements here.

We adjust the definition of backward satisfiability to Def. 5. The definition coincides with the intuition in Fig. 3, except that now ω_F may be encoded into an even larger Ω'_F rather than Ω_F . This relaxation gives more flexibility with no affection on the soundness of the framework, intuitively because excess resources can always be discarded harmlessly in an affine SL.

Definition 5 (Generalized backward satisfiability). A front-end assertion A has *generalized backward satisfiability* w.r.t. the assertion translation $\llbracket \cdot \rrbracket_A$ iff: for any front-end state ω and IVL states Ω , Ω_A , and Ω_F , if $\omega \sim_S \Omega$, $\Omega = \Omega_A \oplus \Omega_F$, and Ω_A satisfies $\llbracket A \rrbracket_A$, then there exist front-end states ω_A and ω_F and IVL state Ω'_F such that: $\omega = \omega_A \oplus \omega_F$, ω_A satisfies A , $\omega_F \sim_S \Omega'_F$, and $\Omega'_F \succeq \Omega_F$.

With Def. 5, Req. 2 from §3.3 needs only technical adaptations:

Requirement 2'. *The translation functions for boolean expressions, input assertions, and primitive statements must satisfy the following properties:*

- The translation $\llbracket b \rrbracket_B$ of any front-end boolean expression $b \in BE_F$ preserves the semantics of b , i.e., for any front-end state $\omega \in \Sigma_F$ and IVL state $\Omega \in \Sigma_V$ with $\omega \sim_S \Omega$, $I_B(b)(\omega)$ is defined iff $\llbracket b \rrbracket_B(\Omega)$ is defined, and they evaluate to the same value when both are defined.
- The translation of input assertions $\llbracket \cdot \rrbracket_{Ap}$ must satisfy:
 - $\llbracket \cdot \rrbracket_{Ap}$ preserves the semantics of all well-formed input assertions in the front-end. Concretely, for any front-end state ω and IVL state Ω with $\omega \sim_S \Omega$, ω satisfies A iff Ω satisfies $\llbracket A \rrbracket_{Ap}$.
 - $\llbracket A \rrbracket_{Ap}$ is monotone for any $A \in A_{Fp}$.
 - $\llbracket A \rrbracket_{Ap}$ of any input front-end assertion A does not contain any free occurrence of P .
 - Any well-formed $A \in A_{Fp}$ has generalized backward satisfiability w.r.t. $\llbracket \cdot \rrbracket_{Ap}$.
- The translation of statements $\llbracket \cdot \rrbracket_{Cp}$ preserves the semantics in the front-end.

To show that our framework for state encoding relations is indeed a generalization, we instantiate it, modeling the state encoding as a function, as stated in [Thm. 6](#).

Theorem 6 (Generalization). *Given an instantiation of the generalized framework where the state encoding is a function, if this instantiation satisfies all requirements from §3.3 then it also satisfies the adapted requirements of the generalized framework.*

Our main result is that any instantiation of the generalized framework that satisfies the adapted requirements is *sound*. That is, [Thm. 4](#), with the adapted requirements, also holds in the generalized framework.

The proof of the theorem requires several adaptations. In the generalized setting, we define the backward translation of IVL semantic assertion P as $\text{btr}(P) \triangleq \{\omega \mid \exists \Omega \in P. \omega \sim_S \Omega\}$, and call P well-formed iff $\text{btr}(P) = \{\omega \mid \forall \Omega. \omega \sim_S \Omega \Rightarrow \Omega \in P\}$. The direction $\text{btr}(A) * \text{btr}(B) \subseteq \text{btr}(A * B)$ in [Lemma 2](#) does not hold for general A and B anymore, but requires B to be well-formed. As a result, Q_0 being well-formed is needed for [Lemma 3](#). Despite these changes, we prove that all the assertions supported by a front-end instantiation of our generalized framework are well-formed as long as the instantiation satisfies all the requirements in §3.5. As a result, the new premise in [Lemma 3](#) is always satisfied, and we get the same soundness result as [Thm. 4](#) in the generalized framework.

The generalized framework provides an expressive foundation for proving the soundness of practical translational verifiers. In particular, it reduces the soundness proof for state encodings to proving a small number of relatively simple requirements. We will demonstrate this expressiveness in §4.4 by instantiating it with an encoding of duplicable permissions for immutable location.

4 Applications of the Framework

In this section, we demonstrate the applicability of our framework by formalizing three front-end translations with different state encodings into Viper, whose state model we formally define in §4.1. We first revisit the encoding from rational permission amount to reals (§4.2), and we show how to extend this encoding to support a backward-satisfiable fragment of magic wands used by Prusti [1]. We also translate a language with lazy field creation (§4.3), and show an encoding of an immutable heap (§4.4). We describe more encodings inspired by real-world front-ends (§4.5). Finally, we discuss previously-reported soundness bugs in existing translational verifiers whose root causes manifest themselves as violations of our framework’s requirements (§4.6).

In all the examples, our framework substantially simplifies the effort required for defining the translation and proving it sound: a formalization without our framework would essentially have to repeat the definitions and proofs of our main theorems, in addition to proving the requirements of our framework. Our framework also makes the soundness proof easy to maintain in practice, as it decomposes the *global* soundness of a front-end translation into *local* proof obligations:

the translation of each input assertion must be backward satisfiable, and the translation of each primitive statement must be sound. This modularity keeps the proof robust under changes: adding a new input assertion (e.g., the sound magic wand fragment in §4.2) does not require revisiting the overall soundness argument, only proving the corresponding requirements for that assertion.

4.1 Viper State Model

The formalization of Viper we use models the heap as a partial function from heap locations to values and the currently-owned permissions as a partial function from heap locations to real numbers, called *permission mask*. A heap h with a permission mask π forms an IDF algebra with the following instantiations: $\Sigma = \{(h, \pi) \in (L \rightarrow V) \times (L \rightarrow [0, 1]) \mid \forall l. \pi(l) > 0 \Rightarrow h(l) \neq \perp\}$, where a heap location $l \in L$ is a pair of an address (modeled as natural number) and a field (identified by a unique name); $\oplus$ adds up the permission amounts at each heap location and combines the values in the heap when the two heaps are compatible (the sum of permissions at each heap location does not exceed 1, and the two heaps do not have different values at any heap location), i.e., $(h_1, \pi_1) \oplus (h_2, \pi_2)$ is defined iff $\forall l. \pi_1(l) + \pi_2(l) \leq 1 \wedge (l \in \text{dom}(h_1) \cap \text{dom}(h_2) \Rightarrow h_1(l) = h_2(l))$, and equals to $(h_1 \cup h_2, \pi_1 + \pi_2)$ when it is defined; $\lfloor _ \rfloor$ clears all the permissions, i.e., $\lfloor (h, \pi) \rfloor \triangleq (h, \lambda l. 0)$; stabilize erases all values for heap locations to which no permission is held, i.e., $\text{stabilize}((h, \pi)) \triangleq (\lambda l. \text{if } \pi(l) > 0 \text{ then } h(l) \text{ else } \perp, \pi)$; stable asserts whether every heap location storing a concrete value has a positive permission amount, i.e., $\text{stable}((h, \pi)) \Leftrightarrow (\forall l. h(l) \neq \perp \Rightarrow \pi(l) > 0)$; unit erases the whole heap with the permissions, i.e., $\text{unit}((h, \pi)) = (\lambda l. \perp, \lambda l. 0)$;

A Viper state is thus an IVL state where the customized heap is instantiated with a heap with a permission mask, i.e., $\Sigma_V = (\text{Var} \rightarrow V) \times \Sigma$ with Σ defined above.

4.2 Encoding Rational Permission Amounts into Real Permission Amounts

In §2.5, we showed that the common encoding of fractional permissions with rational permission amounts into real permission amounts is generally unsound. Our example there made use of magic wands. In this subsection, we show that such an encoding is sound for a front-end with a more restricted assertion language.

To focus on the essentials, we assume that the front-end language is very similar to our IVL, Viper. Both have the same types (e.g., for booleans, integers, and references), values (e.g., **null**), and expressions (except that the front-end uses rational arithmetic whereas the IVL uses reals). Among the primitive statements, local variable assignment and field assignment are common to front-end language and IVL; moreover, the front-end supports allocation and de-allocation, whose translation is discussed below.

We provide the details of the instantiation of our framework in Appendix B.1 and focus on the most interesting aspects here. The front-end models the program heap as a partial map from heap locations (as pairs of natural number identifiers and string-named fields, which is the same as in Viper) to values, and uses rational permission amounts for its ghost heap, i.e., $H_P = \{h_p : L \rightarrow_{\text{fin}} V_F\}$ where $\rightarrow_{\text{fin}}$ represents a finite partial mapping, and $H_F = \{(h, \pi) \in (L \rightarrow V_F) \times (L \rightarrow [0, 1] \cap \mathbb{Q}) \mid \forall l. \pi(l) > 0 \Rightarrow h(l) \neq \perp\}$. The embedding of H_P onto H_F keeps the partial heap, and assigns full permissions to heap locations that contain values and zero permissions otherwise: $\text{embed}_H(h_p) = (h_p, \lambda l. \text{if } h_p(l) \neq \perp \text{ then } 1 \text{ else } 0)$.

Due to the similarity of the front-end language and IVL, the translation functions for types, values, ghost heaps, and expressions are straightforward, except that the type **Rat** is translated to **Real**. $\text{typeInv}(t, x)$ is thus non-trivial only for $t = \text{Rat}$ as $\text{typeInv}(\text{Rat}, x) \triangleq \exists a : \text{Int}. \exists b : \text{Int}. b \neq 0 \wedge x = a/b$.

The input assertion **acc**($x.f, p$) is translated unchanged to **acc**($x.f, p$). Preservation of semantics of the translation and monotonicity of the translated assertion are straightforward. Backward

satisfiability is also intuitive: if $\llbracket p \rrbracket_E^{10}$ evaluates to v under some Ω with $\llbracket \omega_b \rrbracket_S \succeq \Omega$, then it also evaluates to v under $\llbracket \omega_b \rrbracket_S$ (adding resources cannot change a defined evaluation), and hence to v under ω_b in the front-end, so v is rational. The front-end state holding just v permission to $x.f$ (and only the information needed to evaluate x and p) is therefore the smallest state satisfying $\text{acc}(x.f, p)$, and its encoding is smaller than or equal to Ω .

The translations of variable assignment and field assignment are straightforward. Allocation and de-allocation are translated as follows:

$$\begin{aligned} \llbracket x := \text{alloc}(f_1, \dots, f_n) \rrbracket_{Cp} &\triangleq \text{havoc } x; \text{inhale } \text{acc}(x.f_1, 1) * \dots * \text{acc}(x.f_n, 1) \\ \llbracket \text{free}(x.f) \rrbracket_{Cp} &\triangleq \text{exhale } \text{acc}(x.f, 1) \end{aligned}$$

Intuitively, the translation of allocation picks non-deterministically a fresh heap location identifier (**havoc** x) and provides the necessary permissions for the allocated heap locations; the translation of de-allocation removes the permission to the deleted heap location.

We prove in Isabelle/HOL that this instantiation satisfies all requirements of our framework. This instantiation shows that translating rational permission amounts as reals is not per se unsound. If we had added general magic wands as input assertions to our instantiation (as in Fig. 7 in §2.5) then the proof would have failed because Req. 2 would not hold. Nevertheless, our framework can still support a restricted fragment of magic wands.

A sound fragment of magic wands. In practice, the magic wands used by existing verifiers are often far more limited. Prusti [1], for instance, uses only wands whose two sides are *binary assertions* (i.e., never use fractional permissions), and that do not access rational-valued fields. Such assertions are given by the following grammar (we further check that the fields they read do not have the type **Rat** with the framework's well-formedness predicate $\text{wf}_A^{\Delta_F}$), where $e \in BE_F$ and $t \in T_F$:

$$A_b ::= \text{acc}(x.f, 1) \mid e \mid A_b * A_b \mid A_b \vee A_b \mid e \Rightarrow A_b \mid \exists x : t. A_b$$

Each such assertion is translated like the corresponding case of $\llbracket \cdot \rrbracket_A$ in Appendix A; we write $\llbracket \cdot \rrbracket_{Ab}$ for this translation. We then add to the front-end's input assertions all wands $L \multimap R$ whose two sides are binary assertions that do not access rational fields, with the standard interpretation and $\llbracket L \multimap R \rrbracket_{Ap} \triangleq \llbracket L \rrbracket_{Ab} \multimap \llbracket R \rrbracket_{Ab}$. We prove in Isabelle/HOL that $\llbracket L \multimap R \rrbracket_{Ap}$ satisfies Req. 2, so this fragment of magic wands is captured by our framework and, thus, sound.

4.3 Encoding Lazy Field Creation

In §1, we showed an encoding of a Python-like front-end language with lazy field creation into an IVL with static field declarations. In this subsection, we define a front-end that formalizes the lazy field creation feature, and a translation into Viper. Our framework is able to capture this translation to prove its soundness. This instantiation shows, in particular, that our framework is able to express translations with different heap structures (each front-end field is encoded as two IVL fields) and custom proof rules (as we explain below).

We assume a front-end similar to the one from §4.2. Its logic reasons about lazy field creation as follows: It governs access to a location $x.f$ using the standard accessibility predicate $\text{acc}(x.f, p)$. We assume that the front-end can determine statically which fields are *potentially* added to an object, for instance, by using a type system such as Python's MyPy. Allocation provides permission to all those fields and sets their value to **nofield** to indicate that this field has not been created yet. Field updates require permission to access the field, as usual. The first assignment to a field creates the field, which is reflected by replacing **nofield** with the assigned value. Finally, field read requires permission and asserts that the field value is different from **nofield**, as attempting

¹⁰We use $\llbracket \cdot \rrbracket_E$ to denote the translation of general expressions (including non-boolean ones). It is not a parameter of our framework, but framework parameters can use it (like any other mathematical function).

to read a field before it has been added triggers a runtime error. Since **nofield** is not part of the surface syntax, the frontend provides pure input assertion **added**($x.f$) and **missing**($x.f$) to denote $x.f \neq \text{nofield}$ and $x.f = \text{nofield}$, respectively.

To reflect this logic, the front-end provides the designated value **nofield** and uses it in its heap model: $H_P = \{h_p : L \rightarrow_{\text{fin}} V_F \cup \{\text{nofield}\}\}$ and $H_F = \{(h, \pi) \in (L \rightarrow V_F \cup \{\text{nofield}\}) \times (L \rightarrow [0, 1] \cap \mathbb{Q}) \mid \forall l. \pi(l) > 0 \Rightarrow h(l) \neq \perp\}$. The evaluation of a heap access expression $x.f$ is defined only if its value is different from **nofield**. Other than that, the evaluation of expressions is the same as in §4.2.

Our front-end supports the input assertions **acc**($x.f, p$), **added**($x.f$), and **missing**($x.f$). Compared with the interpretation of **acc**($x.f, p$) in the previous subsection, the interpretation here additionally accepts the case where the field is missing, but excludes the case where p evaluates to 0. The interpretation of **added**($x.f$) and **missing**($x.f$) is straightforward. The concrete definition of $I_A^{\Delta_F}$ for the three input assertions is shown in Appendix B.2.

Local variable assignments, allocations, and de-allocations have the same form as in the previous subsection. For simplicity, we restrict field assignments to the form $x.f := v$, where $v \in \text{Var}$ is a local variable. The CSL rules for the primitive statements are the same as before, and so are the operational semantic reduction rules for local variable and field assignments and de-allocations. For an allocation $x := \text{alloc}(f_1, \dots, f_n)$, the allocated locations are initialized to **nofield** rather than arbitrary concrete values. The formal reduction rule is shown in Appendix B.2.

The most interesting aspect of this front-end is its state encoding. Each front-end field f is encoded by *two* IVL fields: The boolean field f_{added} indicates whether the field f has been added, whereas the field f_{IVL} stores the translated value of the front-end heap location. That is, our encoding reflects the cases where a field has been added or not as follows (v is different from **nofield**).

$$\begin{aligned} a.f &\xrightarrow{p} \text{nofield} \text{ for } p \in [0, 1] & a.f_{\text{added}} &\xrightarrow{p} \text{false}, a.f_{\text{IVL}} \xrightarrow{0} \perp \\ a.f &\xrightarrow{p} v \text{ for } p \in [0, 1] & a.f_{\text{added}} &\xrightarrow{p} \text{true}, a.f_{\text{IVL}} \xrightarrow{p} \llbracket v \rrbracket_v \end{aligned}$$

Expressions are translated identically, except that every field name f in field access expressions is replaced by f_{IVL} . The translation of input assertions needs to reflect the above state encoding. In particular, $\llbracket \text{acc}(x.f, p) \rrbracket_A$ denotes permission to f_{added} and, in case the field has been added, also permission to f_{IVL} :

$$\llbracket \text{acc}(x.f, p) \rrbracket_{Ap} \triangleq \llbracket p \rrbracket_E > 0 * \text{acc}(\llbracket x \rrbracket_E.f_{\text{added}}, \llbracket p \rrbracket_E) * (\llbracket x \rrbracket_E.f_{\text{added}} = \text{true} \Rightarrow \text{acc}(\llbracket x \rrbracket_E.f_{\text{IVL}}, \llbracket p \rrbracket_E))$$

Field assignments are translated as follows:

$$\begin{aligned} \llbracket x.f := v \rrbracket_{Cp} &\triangleq \text{if}(\llbracket x \rrbracket_E.f_{\text{added}} = \text{false}) \{ \text{inhale acc}(\llbracket x \rrbracket_E.f_{\text{IVL}}, 1) \} \text{ else } \{ \text{skip} \}; \\ \llbracket x \rrbracket_E.f_{\text{added}} &:= \text{true}; \llbracket x \rrbracket_E.f_{\text{IVL}} := v \end{aligned}$$

Intuitively, it uses the field $\llbracket x \rrbracket_E.f_{\text{added}}$ to check whether this assignment dynamically creates the field. If so, it adds full permission to the field $\llbracket x \rrbracket_E.f_{\text{IVL}}$, which enables future read and write accesses. Moreover, it records in $\llbracket x \rrbracket_E.f_{\text{added}}$ that the field has been created (in either case, as this is a monotonic update), and assigns the value to $\llbracket x \rrbracket_E.f_{\text{IVL}}$. The translation of the other primitive statements is straightforward; the full definition is presented in Appendix B.2.

The front-end translation described above satisfies all requirements of our framework (for state encoding functions) and is, thus, guaranteed to be sound.

4.4 Encoding Immutable Heaps

Our final instantiation illustrates how our framework supports duplicable resources into an IVL that offers only (non-duplicable) fractional resources. Duplicable resources have many applications, for instance, as permission to acquire a lock or to represent monotonic information. In this subsection, we use access to immutable data as an example for duplicable resources.

To avoid the complications of initializing immutable fields, we assume a front-end here, where immutable data is stored in a separate heap. Field accesses select the heap using different dereferencing operators, $x.f$ for the mutable heap and $x_{\circ}f$ for the immutable heap.¹¹ Allocation in the mutable heap works as usual, whereas allocation in the immutable heap takes as arguments the initial values of the fields and initializes them. Hence, regular allocation provides the usual ownership of the allocated locations, donated again by $\text{acc}(x.f, p)$, whereas immutable allocation provides a *duplicable* resource $\text{init}(x_{\circ}f)$, which permits read access, but not assignments.

To reflect these ideas, our front-end is similar to the one in §4.2, but has two heaps: $H_P = \{(h, h_{im}) \in (L \rightarrow V_F) \times (L \rightarrow V_F)\}$, where the first component is the mutable partial heap and the second one is the immutable heap. The ghost heap set is $H_F = \{(h, \pi, h_{im}) \in (L \rightarrow V_F) \times (L \rightarrow [0, 1] \cap \mathbb{Q}) \times (L \rightarrow V_F) \mid \forall l. \pi(l) > 0 \Rightarrow h(l) \neq \perp\}$. The embedding function embed_H maps the immutable heap h_{im} identically and the mutable heap h as to a heap (h, π) with rational permissions. The resource of each heap location in the immutable heap is stable and duplicable, *i.e.*, the immutable heap in H_F is instantiated as an IDF algebra with the following definitions of the operations:

- $h_1 \oplus h_2$ is defined iff $\forall l \in L$, if both $h_1(l)$ and $h_2(l)$ are defined then $h_1(l) = h_2(l)$. In this case, $(h_1 \oplus h_2)(l) \triangleq$ if $h_1(l)$ is defined then $h_1(l)$ else $h_2(l)$.
- $|h| = \text{stabilize}(h) \triangleq h$, and $\text{stable}(h)$ holds for any h .
- $\text{unit}(h) \triangleq \text{emp}$, where emp is the empty heap $\lambda l. \perp$.

The front-end supports the same primitive statements as in §4.2. In addition, the statement $x := \text{imalloc}(f := v)$ allocates and initializes an immutable location $x_{\circ}f$. The operational semantic reduction rule and CSL rule for this statement are straightforward; we list them in Appendix B.3.

Since our IVL does not provide duplicable resources, we encode them as arbitrary positive fractional permission amounts. As a result, the representation for a duplicable front-end resource is not unique. Our generalized framework allows us to express such state encoding relations.

The encoding of front-end ghost heaps is modeled as a binary relation. Both front-end heaps are encoded into a single IVL heap. To disambiguate accesses, we use a map f_h that maps each front-end field f in the mutable heap to the IVL field hf ; the map f_v analogously maps each front-end field g in the immutable heap to the IVL field vg . The ranges of f_h and f_v are disjoint. The encoding of the mutable heap is the same as in §4.2 (except that here field names are mapped). The (allocated) heap locations in the immutable heap are encoded to having *any* permission $p \in (0, 1)$; that is, the state relation permits *all* such amounts p . Un-allocated locations have zero permission and no value.

To formalize the state encoding relation, we need to divide the IVL heap into the parts corresponding to the mutable and immutable front-end heaps. In those parts, we then undo the field name mappings f_h and f_v so that we can relate them to the front-end fields. Concretely, we define $H|_{f_h}$ to be the IVL sub-heap constructed by taking only locations with field names in the range of f_h ; we construct $H|_{f_v}$ analogously. H_{rem} denotes the remaining part of H after removing $H|_{f_h}$ and $H|_{f_v}$. $\Pi|_{f_h}$, $\Pi|_{f_v}$, and Π_{rem} are defined similarly for the permission mask Π (while “removing” means setting the permission to each location in the corresponding part to zero). Now we can define the state relation, by reusing the ghost heap encoding $\llbracket \cdot \rrbracket_H$ from §4.2 for the mutable heap:

$$\begin{aligned} (h, \pi, h_{im}) \sim_H (H, \Pi) &\triangleq (H|_{f_h}, \Pi|_{f_h}) = \llbracket (h, \pi) \rrbracket_H \wedge H_{\text{rem}} = \lambda l. \perp \wedge \Pi_{\text{rem}} = \lambda l. 0 \wedge (\forall l. \\ &(h_{im}(l) = \perp \Rightarrow (H|_{f_v}(l) = \perp \wedge \Pi|_{f_v}(l) = 0)) \wedge \\ &(h_{im}(l) = v \Rightarrow (H|_{f_v}(l) = \llbracket v \rrbracket_V \wedge \Pi|_{f_v}(l) \in (0, 1)))) \end{aligned}$$

Heap access expressions are translated accordingly: the field names for all accesses to the mutable heap (of the form $x.f$) are translated by f_h , while those for the accesses to the immutable heap (of

¹¹Note that both heaps share the same identifiers, so a field f may occur in both heaps, and a syntactic distinction of the two accesses is necessary.

the form $x_o f$) are translated by f_o . The translation for the input assertions are as follows, where $\text{acc}(x.f, _)$ represents ownership of an arbitrary positive fractional permission amount for $x.f$.

$$\llbracket \text{acc}(x.f, p) \rrbracket_{\text{Ap}} \triangleq \text{acc}(\llbracket x \rrbracket_E . f_h(f), \llbracket p \rrbracket_E) \quad \llbracket \text{init}(x_o f) \rrbracket_{\text{Ap}} \triangleq \text{acc}(\llbracket x \rrbracket_E . f_o(f), _)$$

The translation of local variable assignments, allocations in the mutable heap, and de-allocations is similar to that in §4.2; we show them in Appendix B.3. The translation of allocation in the immutable heap assigns a fresh heap location, adds the resource corresponding to $\text{init}(x_o f)$ to the state, and assumes that the new location has its initial value:

$$\llbracket x := \text{immalloc}(f := v) \rrbracket_{\text{Cp}} \triangleq \text{havoc } x; \text{inhale } \text{acc}(x.f_o(f), _) * x.f_o(f) = v$$

We proved in Isabelle/HOL that the instantiation of our *generalized* framework for immutable data satisfies all requirements and is, thus, guaranteed to be sound. This instantiation also demonstrates that our generalized framework is sufficiently expressive to handle complex, but common state encodings such as translating duplicable resources to non-duplicable fractional resources.

4.5 Additional Encodings Inspired by Real-World Front-Ends

Beyond the three instantiations formalized above, we believe that our framework can capture many other front-ends. To illustrate this point, we outline three further state encodings, inspired by real-world front-ends, that also fit our framework:

The Pancake verifier’s state model [60]: The front-end heap is a fixed-size array of cells, encoded into a Viper sequence heap of references, where the cell at offset i corresponds to the heap location $\text{heap}[i]$ refers to. It uses the same fractional permission model as Viper.

The block-offset memory model for C, adapted from Leroy [30]: C memory is modeled as an unbounded but finite set of blocks, each with a unique identifier, a linear array of cells (with binary permissions) indexed by their offset, and a fixed size. The encoding maps each cell (identified by its block and offset) to a unique heap location storing its value in a `val` field, and each block to a heap location whose `size` field stores the block’s size.

The object-oriented memory model from Lööw et al. [32]: The front-end heap consists of an unbounded but finite set of objects, each with a unique identifier, a list of cells (with binary permissions) named by field, and a domain (the set of its field names). The encoding maps each object to a unique heap location, each cell to the field of that location with the same name, and stores the object’s domain in an additional field.

In all three instantiations, the homomorphism requirement holds: Two heaps in the Pancake state model must agree on the array size and the content of each cell, allowing the combination of the encodings; the permissions are simply added for each cell. In the other two models, two states with the same identifier must agree on their shared contents and have disjoint domains on the non-duplicable contents, so their encodings combine: two C blocks must agree on their size and two objects on their domain; blocks or objects from two states must own disjoint cells. Backward satisfiability of the input assertions (e.g., fractional ownership of a cell, or a block’s size) is likewise straightforward.

4.6 Identifying Unsoundness in Real-World Front-End Translations

In addition to proving existing front-ends sound, our framework captures common sources of unsoundness in existing verifiers: the root causes of several previously-reported soundness bugs correspond to violations of our requirements, showing that checking these requirements could have caught them. For example, issues #1189 [47] and #1518 [48] in Prusti [1] are caused by passing the same shared reference as two arguments to a method call, which creates two independent fractional permissions in the IVL, whereas the front-end state model forbids such a split, which thus violates the homomorphism requirement in Req. 1. Violations of Req. 2 underlie issue #955 [53]

(an unsound quantifier-simplification rewrite) in VerCors [4], issue #1501 [49] (wrong encoding of matches! inside assertions) in Prusti, and issue #281 [50] (unsound bool/object round-trip cast) in Nagini [14]. Violations of requirements in Appendix A underlie further bugs: soundness of the front-end SL rules is violated by issue #703 [52] (an unreachable send is treated as transferring permission) in VerCors, injectivity of value translation by issue #1357 [54] (int and short are both mapped to the same IVL type) in VerCors, and soundness of the statement translation by issue #161 [51] (static field updates are not propagated) in Nagini.

5 Related Work

Our work was inspired by Dardinier et al. [11]’s framework for proving the soundness of SL-based translational verifiers. However, their framework requires the front-end and the IVL to have the same state model, which is not the case in existing translational SL-verifiers, such as Gobra, Nagini [14], a Pancake verifier [60], Prusti, and VeriFast. In contrast, our framework supports complex state encodings, both in terms of the representation of values and of SL resources.

Several existing works target the soundness of front-end translations. For instance, Summers and Müller [46] and Wolf et al. [59] sketch soundness proofs for the Viper translations of the RSL weak memory logic [56] and the TaDA logic [9], respectively. Both involve non-standard SL resources, for which our framework provides a formal foundation.

Instead of translating state models to the IVL, Gillian uses an IVL and verification back-end that are parametric in the state model. This simplifies front-end soundness proofs but requires the back-end to be sound for all instantiations. Löw et al. [32] present a framework for such proofs in Rocq, which is parametric in the concrete and symbolic state models, with sufficient conditions for a sound instantiation. However, unlike our work, it includes no statement or assertion translations and supports neither loops nor concurrency, so it is unclear whether it can capture realistic front-end translations. Moreover, it does not facilitate soundness proofs for IVLs that are not parametric in the state model, such as Raven, VeriFast, and Viper.

Front-end translations to IVLs not based on SL [57, 3, 20, 16, 37] involves generally much simpler state encodings. In fact, the existing unsound encodings from §2.2 and §2.5 as well as our sufficient soundness conditions are all concerned with the representation of SL resources.

Some existing approaches target the soundness of SL-based verification back-ends. Parthasarathy et al. [37] present a technique for automatically producing certificates for successful verification runs in Viper. Zimmerman et al. [61] present a formalization of a variant of Viper’s symbolic execution back-end for gradual verification. Jacobs et al. [23] formalize and prove sound a simplified version of the symbolic execution back-end of VeriFast [22] in Rocq. These back-end soundness proofs can be combined with our framework’s front-end proofs for an overall soundness guarantee.

6 Conclusion and Future Work

We presented the first framework, formalized in Isabelle/HOL, for proving the soundness of the front-ends of translational separation logic verifiers with complex state encodings, which are pervasive in existing verifiers. Our sufficient soundness conditions greatly simplify soundness proofs and help uncover flawed encodings quickly, as we illustrate on several common encodings.

As future work, we plan to extend the framework to support additional concurrency primitives (e.g., threads and locks) and to apply the framework to prove soundness of existing front-ends.

Data Availability Statement

All technical results presented in this paper have been formalized and proved in Isabelle/HOL. The formalization is publicly available in our artifact [31]. The latest version is available at <https://doi.org/10.5281/zenodo.21509501>.

References

- [1] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. *Proc. ACM Program. Lang.* 3, OOPSLA (Oct. 2019), 147:1–147:30. doi:10.1145/3360573
- [2] Sacha-Élie Ayoun, Xavier Denis, Petar Maksimović, and Philippa Gardner. 2025. A Hybrid Approach to Semi-Automated Rust Verification. arXiv:2403.15122 [cs] doi:10.48550/arXiv.2403.15122
- [3] Michael Backes, Catalin Hritcu, and Thorsten Tarrach. 2011. Automatically Verifying Typing Constraints for a Data Processing Language. In *Certified Programs and Proofs (CPP)*, Jean-Pierre Jouannaud and Zhong Shao (Eds.). doi:10.1007/978-3-642-25379-9_22
- [4] Stefan Blom, Saeed Darabi, Marieke Huisman, and Wytse Oortwijn. 2017. The VerCors Tool Set: Verification of Parallel and Concurrent Software. In *Integrated Formal Methods*, Nadia Polikarpova and Steve Schneider (Eds.). Springer International Publishing, Cham, 102–110. doi:10.1007/978-3-319-66845-1_7
- [5] Richard Bornat, Cristiano Calcagno, Peter O’Hearn, and Matthew Parkinson. 2005. Permission Accounting in Separation Logic. *SIGPLAN Not.* 40, 1 (Jan. 2005), 259–270. doi:10.1145/1047659.1040327
- [6] John Boyland. 2003. Checking Interference with Fractional Permissions. In *Static Analysis*, Radhia Cousot (Ed.). Springer, Berlin, Heidelberg, 55–72. doi:10.1007/3-540-44898-5_4
- [7] Joshua M. Cohen and Philip Johnson-Freyd. 2024. A Formalization of Core Why3 in Coq. *Proc. ACM Program. Lang.* 8, POPL, Article 60 (jan 2024), 30 pages. doi:10.1145/3632902
- [8] Pascal Cuoq, Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. 2012. Frama-C – A Software Analysis Perspective. In *SEFM (Lecture Notes in Computer Science, Vol. 7504)*. Springer, 233–247. doi:10.1007/978-3-642-33826-7_16
- [9] Pedro da Rocha Pinto, Thomas Dinsdale-Young, and Philippa Gardner. 2014. TaDA: A Logic for Time and Data Abstraction. In *European Conference on Object-Oriented Programming (ECOOP) (Lecture Notes in Computer Science, Vol. 8586)*, Richard E. Jones (Ed.). Springer, 207–231. doi:10.1007/978-3-662-44202-9_9
- [10] Thibault Dardinier, Peter Müller, and Alexander J. Summers. 2022. Fractional Resources in Unbounded Separation Logic. *Proc. ACM Program. Lang.* 6, OOPSLA2 (Oct. 2022), 163:1066–163:1092. doi:10.1145/3563326
- [11] Thibault Dardinier, Michael Sammler, Gaurav Parthasarathy, Alexander J. Summers, and Peter Müller. 2025. Formal Foundations for Translational Separation Logic Verifiers. *Proc. ACM Program. Lang.* 9, POPL (Jan. 2025), 569–599. doi:10.1145/3704856
- [12] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM (Lecture Notes in Computer Science, Vol. 13478)*. Springer, 90–105. doi:10.1007/978-3-031-17244-1_6
- [13] Thomas Dinsdale-Young, Mike Dodds, Philippa Gardner, Matthew J. Parkinson, and Viktor Vafeiadis. 2010. Concurrent Abstract Predicates. In *ECOOP 2010 – Object-Oriented Programming*, Theo D’Hondt (Ed.). Springer, Berlin, Heidelberg, 504–528. doi:10.1007/978-3-642-14107-2_24
- [14] Marco Eilers and Peter Müller. 2018. Nagini: A Static Verifier for Python. In *Computer Aided Verification*, Hana Chockler and Georg Weissenbacher (Eds.). Springer International Publishing, Cham, 596–603. doi:10.1007/978-3-319-96145-3_33
- [15] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 - Where Programs Meet Provers. In *ESOP (Lecture Notes in Computer Science, Vol. 7792)*. Springer, 125–128. doi:10.1007/978-3-642-37036-6_8
- [16] Jean Fortin. 2013. *BSP-Why, un outil pour la vérification déductive de programmes BSP : machine-checked semantics and application to distributed state-space algorithms. (BSP-Why, a tool for deductive verification of BSP programs : sémantiques mécanisées et application aux algorithmes d’espace d’états distribués)*. Ph. D. Dissertation. University of Paris-Est, France. <https://tel.archives-ouvertes.fr/tel-00974977>
- [17] José Fragoso Santos, Petar Maksimović, Sacha-Élie Ayoun, and Philippa Gardner. 2020. Gillian, part i: a multi-language platform for symbolic execution. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 927–942. doi:10.1145/3385412.3386014
- [18] Alexey Gotsman, Josh Berdine, Byron Cook, Noam Rinetzk, and Mooly Sagiv. 2007. Local reasoning for storable locks and threads. In *Asian Symposium on Programming Languages And Systems*. Springer, 19–37. doi:10.1007/978-3-540-76637-7_3
- [19] Ekanshdeep Gupta, Nisarg Patel, and Thomas Wies. 2025. Raven: An SMT-Based Concurrency Verifier. In *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 15931)*, Ruzica Piskac and Zvonimir Rakamaric (Eds.). Springer, 81–106. doi:10.1007/978-3-031-98668-0_4
- [20] Paolo Herms. 2013. *Certification of a Tool Chain for Deductive Program Verification. (Certification d’une chaîne de vérification déductive de programmes)*. Ph. D. Dissertation. University of Paris-Sud, Orsay, France. <https://tel.archives-ouvertes.fr/tel-00789543>

- [21] Bart Jacobs, Dragan Bosnacki, and Ruurd Kuiper. 2015. Modular Termination Verification. In *29th European Conference on Object-Oriented Programming (ECOOP 2015) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 37)*, John Tang Boyland (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 664–688. doi:10.4230/LIPIcs.ECOOP.2015.664
- [22] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods (NFM) (Lecture Notes in Computer Science, Vol. 6617)*, Mihaela Gheorghiu Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi (Eds.). Springer, 41–55. doi:10.1007/978-3-642-20398-5_4
- [23] Bart Jacobs, Frédéric Vogels, and Frank Piessens. 2015. Featherweight VeriFast. *Logical Methods in Computer Science* Volume 11, Issue 3 (Sept. 2015). doi:10.2168/LMCS-11(3:19)2015
- [24] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '15)*. Association for Computing Machinery, New York, NY, USA, 637–650. doi:10.1145/2676726.2676980
- [25] Bernhard Kragl and Shaz Qadeer. 2021. The Civi Verifier. In *#PLACEHOLDER_PARENT_METADATA_VALUE#*. TU Wien Academic Press, 143–152. doi:10.34727/2021/isbn.978-3-85448-046-4_23
- [26] Robbert Krebbers and Freek Wiedijk. 2013. Separation logic for non-local control flow and block scope variables. In *International Conference on Foundations of Software Science and Computational Structures*. Springer, 257–272. doi:10.1007/978-3-642-37075-5_17
- [27] K. Rustan M. Leino. 2008. This Is Boogie 2. (June 2008). Available from <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/krm178.pdf>.
- [28] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer, Berlin, Heidelberg, 348–370. doi:10.1007/978-3-642-17511-4_20
- [29] K. Rustan M. Leino and Peter Müller. 2009. A Basis for Verifying Multi-threaded Programs. In *Programming Languages and Systems*, Giuseppe Castagna (Ed.). Springer, Berlin, Heidelberg, 378–393. doi:10.1007/978-3-642-00590-9_27
- [30] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. doi:10.1145/1538788.1538814
- [31] Hongyi Ling, Thibault Dardinier, Ellen Arlt, and Peter Müller. 2026. *Sound State Encodings in Translational Separation Logic Verifiers (Artifact)*. doi:10.5281/zenodo.21509502
- [32] Andreas Löw, Seung Hoon Park, Daniele Nantes-Sobrinho, Sacha-Élie Ayoun, Opale Sjöstedt, and Philippa Gardner. 2025. Compositional Symbolic Execution for the Next 700 Memory Models. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 2815–2842. doi:10.1145/3763151
- [33] Petar Maksimović, Sacha-Élie Ayoun, José Fragoso Santos, and Philippa Gardner. 2021. Gillian, Part II: Real-World Verification for JavaScript and C. In *Computer Aided Verification*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer International Publishing, Cham, 827–850. doi:10.1007/978-3-030-81688-9_38
- [34] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. 2016. Viper: A Verification Infrastructure for Permission-Based Reasoning. In *Verification, Model Checking, and Abstract Interpretation*, Barbara Jobstmann and K. Rustan M. Leino (Eds.). Springer, Berlin, Heidelberg, 41–62. doi:10.1007/978-3-662-49122-5_2
- [35] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. 2002. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, Vol. 2283. Springer. doi:10.1007/3-540-45949-9
- [36] Peter W. O’Hearn. 2004. Resources, Concurrency and Local Reasoning. In *CONCUR 2004 - Concurrency Theory*, Philippa Gardner and Nobuko Yoshida (Eds.). Springer, Berlin, Heidelberg, 49–67. doi:10.1007/978-3-540-28644-8_4
- [37] Gaurav Parthasarathy, Thibault Dardinier, Benjamin Bonneau, Peter Müller, and Alexander J. Summers. 2024. Towards Trustworthy Automated Program Verifiers: Formally Validating Translations into an Intermediate Verification Language. *Proc. ACM Program. Lang.* 8, PLDI, Article 208 (jun 2024), 25 pages. doi:10.1145/3656438
- [38] Gaurav Parthasarathy, Peter Müller, and Alexander J. Summers. 2021. Formally Validating a Practical Verification Condition Generator. In *Computer Aided Verification (CAV) (LNCS, Vol. 12760)*, Alexandra Silva and K. Rustan M. Leino (Eds.). 704–727. doi:10.1007/978-3-030-81688-9_33
- [39] Willem Penninckx, Bart Jacobs, and Frank Piessens. 2015. Sound, Modular and Compositional Verification of the Input/Output Behavior of Programs. In *Programming Languages and Systems*, Jan Vitek (Ed.). Springer, Berlin, Heidelberg, 158–182. doi:10.1007/978-3-662-46669-8_7
- [40] João Pereira, Tobias Klenze, Sofia Giampietro, Markus Limbeck, Dionysios Spiliopoulos, Felix Wolf, Marco Eilers, Christoph Sprenger, David Basin, Peter Müller, and Adrian Perrig. 2025. Protocols to Code: Formal Verification of a Secure Next-Generation Internet Router. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (New York, NY, USA, 2025-11-22) (CCS '25)*. Association for Computing Machinery, 1469–1483. doi:10.1145/3719027.3765104

- [41] João C. Pereira, Isaac van Bakel, Patricia Firlejczyk, Marco Eilers, and Peter Müller. 2025. Modular Reasoning about Global Variables and Their Initialization. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 2310–2337. doi:10.1145/3763133
- [42] J.C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. 55–74. doi:10.1109/LICS.2002.1029817
- [43] Jan Smans, Bart Jacobs, and Frank Piessens. 2012. Implicit Dynamic Frames. *ACM Trans. Program. Lang. Syst.* 34, 1 (May 2012), 2:1–2:58. doi:10.1145/2160910.2160911
- [44] Simon Spies, Niklas Mück, Haoyi Zeng, Michael Sammler, Andrea Lattuada, Peter Müller, and Derek Dreyer. 2025. Destabilizing Iris. *Proc. ACM Program. Lang.* 9, PLDI (2025), 848–873. doi:10.1145/3729284
- [45] Christoph Sprenger, Tobias Klenze, Marco Eilers, Felix A. Wolf, Peter Müller, Martin Clochard, and David Basin. 2020. Igloo: soundly linking compositional refinement and separation logic for distributed system verification. *Artifact for "Igloo: Soundly Linking Compositional Refinement and Separation Logic for Distributed System Verification"* 4, OOPSLA (Nov. 2020), 152:1–152:31. doi:10.1145/3428220
- [46] Alexander J. Summers and Peter Müller. 2020. Automating deductive verification for weak-memory programs (extended version). *International Journal on Software Tools for Technology Transfer (STTT)* 22, 6 (2020), 709–728. doi:10.1007/S10009-020-00559-Y
- [47] GitHub user: Aurel300. 2022. Shared reference unsoundness (issue #1189). <https://github.com/viperproject/prusti-dev/issues/1189>.
- [48] GitHub user: cliu369. 2024. Assert(false) passing with Shared References (issue #1518). <https://github.com/viperproject/prusti-dev/issues/1518>.
- [49] GitHub user: fpoli. 2024. Wrong encoding of signed discriminants in pure code (issue #1501). <https://github.com/viperproject/prusti-dev/issues/1501>.
- [50] GitHub user: gewlar. 2026. Unsound behavior when casting to object (issue #281). <https://github.com/marcoeilers/nagini/issues/281>.
- [51] GitHub user: marcoeilers. 2024. Unsoundness relating to static fields (issue #161). <https://github.com/marcoeilers/nagini/issues/161>.
- [52] GitHub user: pieter-bos. 2021. Send/Recv is unsound where guard checking is incomplete (issue #703). <https://github.com/utwente-fmt/vercors/issues/703>.
- [53] GitHub user: pieter-bos. 2023. SimplifyNestedQuantifiers makes an unsound rewrite (issue #955). <https://github.com/utwente-fmt/vercors/issues/955>.
- [54] GitHub user: superaxander. 2025. Unsoundness due to violation of effective type rules (issue #1357). <https://github.com/utwente-fmt/vercors/issues/1357>.
- [55] Viktor Vafeiadis. 2011. Concurrent Separation Logic and Operational Semantics. *Electronic Notes in Theoretical Computer Science* 276 (Sept. 2011), 335–351. doi:10.1016/j.entcs.2011.09.029
- [56] Viktor Vafeiadis and Chinmay Narayan. 2013. Relaxed separation logic: a program logic for C11 concurrency. In *Object Oriented Programming Systems Languages & Applications, (OOPSLA)*, Antony L. Hosking, Patrick Th. Eugster, and Cristina V. Lopes (Eds.). doi:10.1145/2509136.2509532
- [57] Frédéric Vogels, Bart Jacobs, and Frank Piessens. 2009. A Machine Checked Soundness Proof for an Intermediate Verification Language. In *Theory and Practice of Computer Science, Conference on Current Trends in Theory and Practice of Computer Science (SOFSEM) (Lecture Notes in Computer Science, Vol. 5404)*, Mogens Nielsen, Antonín Kucera, Peter Bro Miltersen, Catuscia Palamidessi, Petr Tuma, and Frank D. Valencia (Eds.). Springer, 570–581. doi:10.1007/978-3-540-95891-8_51
- [58] Felix A. Wolf, Linard Arqunt, Martin Clochard, Wytse Oortwijn, João C. Pereira, and Peter Müller. 2021. Gobra: Modular Specification and Verification of Go Programs. In *Computer Aided Verification*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer International Publishing, Cham, 367–379. doi:10.1007/978-3-030-81685-8_17
- [59] Felix A. Wolf, Malte Schwerhoff, and Peter Müller. 2022. Concise outlines for a complex logic: a proof outline checker for TaDA. *Formal Methods in System Design* 61, 1 (2022), 110–136. doi:10.1007/S10703-023-00427-W
- [60] Junming Zhao, Miki Tanaka, Johannes Aman Pohjola, Alessandro Legnani, Tiana Tsang Ung, H. Truong, Tsun Wang Sau, Thomas Sewell, Rob Sison, Hira Syeda, Magnus Myreen, Michael Norrish, and Gernot Heiser. 2025. Verifying Device Drivers with Pancake. arXiv:2501.08249 [cs.PL] <https://arxiv.org/abs/2501.08249>
- [61] Conrad Zimmerman, Jenna DiVincenzo, and Jonathan Aldrich. 2024. Sound Gradual Verification with Symbolic Execution. *Proc. ACM Program. Lang.* 8, POPL (Jan. 2024), 85:2547–85:2576. doi:10.1145/3632927

A Full Definitions and Semantics in the Framework

The axioms for IDF algebra is listed in Fig. 10.

$$\begin{aligned}
a \oplus b &= b \oplus a & a \oplus (b \oplus c) &= (a \oplus b) \oplus c & c &= a \oplus b \wedge c = c \oplus c \Rightarrow a = a \oplus a & x &= x \oplus |x| \\
|x| &= |x| \oplus |x| & x &= x \oplus c \Rightarrow |x| \succeq c & |a \oplus b| &= |a| \oplus |b| & a &= b \oplus x \wedge a = b \oplus y \wedge |x| = |y| \Rightarrow x = y \\
\text{stable}(\omega) &\Rightarrow \omega = \text{stabilize}(\omega) & \text{stable}(\text{stabilize}(\omega)) & & \text{stabilize}(a \oplus b) &= \text{stabilize}(a) \oplus \text{stabilize}(b) \\
x &= \text{stabilize}(x) \oplus |x| & a &= b \oplus \text{unit}(c) \Rightarrow a = b & a &= a \oplus \text{unit}(a) & |\text{unit}(a)| &= \text{unit}(a) \\
& & \text{stable}(\text{unit}(a)) & & & & &
\end{aligned}$$

Fig. 10. Axioms for the IDF algebra $(\Sigma, \oplus, |_, \text{stable}, \text{stabilize}, \text{unit})$. Define $\omega' \succeq \omega \triangleq (\exists r. \omega' = \omega \oplus r)$.

The axiomatic semantic rules for IVL statements **havoc** x , **inhale** A , **exhale** A , $C_1; C_2$, and **if**(b) $\{C_1\}$ **else** $\{C_2\}$ are listed in Fig. 11.

$$\begin{array}{c}
\text{INHALEX} \\
\frac{\text{selfFraming}(P) \quad \text{frames}(A, P)}{\Delta \vdash [P] \text{inhale } A [P * A]} \\
\\
\text{HAVOCAX} \\
\frac{\text{selfFraming}(P) \quad \Delta(x) = \tau}{\Delta \vdash [P] \text{havoc } x [\exists x \in \tau. P]} \\
\\
\text{EXHALEX} \\
\frac{\text{selfFraming}(P) \quad P \subseteq Q * A \quad \text{selfFraming}(Q)}{\Delta \vdash [P] \text{exhale } A [Q]} \\
\\
\text{SEQAX} \\
\frac{\Delta \vdash [P] C_1 [R] \quad \Delta \vdash [R] C_2 [Q]}{\Delta \vdash [P] C_1; C_2 [Q]} \\
\\
\text{IFAX} \\
\frac{\text{selfFraming}(P) \quad \text{frames}(P, b) \quad \Delta \vdash [P * b] C_1 [B_1] \quad \Delta \vdash [P * \neg b] C_2 [B_2]}{\Delta \vdash [P] \text{if}(b) \{C_1\} \text{else } \{C_2\} [B_1 \vee B_2]}
\end{array}$$

Fig. 11. Axiomatic semantics for the IVL statements used in our framework.

The extension of $\text{wf}_A^{\Delta_F}$ for front-end assertions is defined via an auxiliary function $\text{wf}_A^{\Delta_F}(A)_B$, where B is a boolean indicator of whether A is in the scope of any μ binder. $\text{wf}_A^{\Delta_F}(A)_B$ is defined as follows, where $\Delta_F[x := t]$ represents the type context derived from Δ_F by replacing the type of local variable x to t , and $\text{wf}_A^{\Delta_F}(A) \triangleq \text{wf}_A^{\Delta_F}(A)_{\text{false}}$.

$$\begin{aligned}
\text{wf}_A^{\Delta_F}(A_p)_B &\triangleq \text{wf}_A^{\Delta_F}(A_p) \\
\text{wf}_A^{\Delta_F}(A_1 * A_2)_B &\triangleq \text{wf}_A^{\Delta_F}(A_1)_B \wedge \text{wf}_A^{\Delta_F}(A_2)_B \\
\text{wf}_A^{\Delta_F}(A_1 \vee A_2)_B &\triangleq \text{wf}_A^{\Delta_F}(A_1)_B \wedge \text{wf}_A^{\Delta_F} A_2 B \\
\text{wf}_A^{\Delta_F}(\exists x : t. A)_B &\triangleq \text{wf}_A^{\Delta_F[x:=t]}(A)_B \\
\text{wf}_A^{\Delta_F}(b \Rightarrow A)_B &\triangleq \text{wf}_A^{\Delta_F}(A)_B \\
\text{wf}_A^{\Delta_F}(b)_B &\triangleq \text{true} \\
\text{wf}_A^{\Delta_F}(P)_B &\triangleq B \\
\text{wf}_A^{\Delta_F}(\mu P. A)_B &\triangleq \text{wf}_A^{\Delta_F}(A)_{\text{true}}
\end{aligned}$$

To define the extension of interpretation function $I_A^{\Delta_F}(\cdot)$ and free variable function fv_A for front-end assertions, we first define auxiliary interpretation $I_A^{\Delta_F}(\cdot)_\Gamma$ parameterized by the interpretation

Γ for predicate symbol P and auxiliary function $\text{fv}_A(\cdot)_F$ parameterized by the free variable set F of predicate symbol P as follows, where $\omega[v/x]$ represents the resulting state of overwriting the value of local variable x in ω to v , and $\text{LFP}(\lambda S. f(S)) \triangleq \bigcap_{S: f(S) \subseteq S} S$ is the least fixed point of f when it monotonely increases on S , which we prove is the case for $I_A^{\Delta F}(\cdot)_\Gamma$ and $\text{fv}_A(A)_F$. Then $I_A^{\Delta F}(A) \triangleq I_A^{\Delta F}(A)_\emptyset$ and $\text{fv}_A(A) \triangleq \text{fv}_A(A)_\emptyset$.

$$\begin{aligned}
I_A^{\Delta F}(A_1 * A_2)_\Gamma &\triangleq \{\omega \mid \exists \omega_1 \in I_A^{\Delta F}(A_1)_\Gamma, \omega_2 \in I_A^{\Delta F}(A_2)_\Gamma. \omega = \omega_1 \oplus \omega_2\} \\
I_A^{\Delta F}(A_1 \vee A_2)_\Gamma &\triangleq I_A^{\Delta F}(A_1)_\Gamma \cup I_A^{\Delta F}(A_2)_\Gamma \\
I_A^{\Delta F}(\exists x : t. A)_\Gamma &\triangleq \{\omega \mid \exists v. \text{tp}(v) = t \wedge \omega[v/x] \in I_A^{\Delta F}[x:=t](A)_\Gamma\} \\
I_A^{\Delta F}(b \Rightarrow A)_\Gamma &\triangleq \left\{ \omega \mid I_B(b)(\omega) \text{ is defined } \wedge \left(I_B(b)(\omega) = \text{true} \Rightarrow \omega \in I_A^{\Delta F}(A)_\Gamma \right) \right\} \\
I_A^{\Delta F}(b)_\Gamma &\triangleq \{\omega \mid I_B(b)(\omega) = \text{true}\} \\
I_A^{\Delta F}(P)_\Gamma &\triangleq \Gamma \\
I_A^{\Delta F}(\mu P. A)_\Gamma &\triangleq \text{LFP} \left(\lambda S. I_A^{\Delta F}(A)_S \right) \\
\text{fv}_A(A_1 * A_2)_F &\triangleq \text{fv}_A(A_1)_F \cup \text{fv}_A(A_2)_F \\
\text{fv}_A(A_1 \vee A_2)_F &\triangleq \text{fv}_A(A_1)_F \cup \text{fv}_A(A_2)_F \\
\text{fv}_A(\exists x : t. A)_F &\triangleq \text{fv}_A(A)_F \setminus \{x\} \\
\text{fv}_A(b \Rightarrow A)_F &\triangleq \text{fv}_B(b) \cup \text{fv}_A(A)_F \\
\text{fv}_A(b)_F &\triangleq \text{fv}_B(b) \\
\text{fv}_A(P)_F &\triangleq F \\
\text{fv}_A(\mu P. A)_F &\triangleq \text{LFP}(\lambda S. \text{fv}_A(A)_S)
\end{aligned}$$

In the Isabelle/HOL formalization, we use de Bruijn indices, and prove the following overapproximation of $\text{fv}_A(\mu P. A)$.

$$\text{fv}_A(\mu P. A) \subseteq \bigcup_{x \in \text{fv}_A(A)} \{0, 1, \dots, x\}$$

The extension of wf_C , fv_C , and wr to all compositions of front-end statements is as follows.

$$\begin{aligned}
\text{wf}_C^{\Delta F}(\text{skip}) &\triangleq \text{true} \\
\text{wf}_C^{\Delta F}(c_1; c_2) &\triangleq \text{wf}_C^{\Delta F}(c_1) \wedge \text{wf}_C^{\Delta F}(c_2) \\
\text{wf}_C^{\Delta F}(\text{if}(b) \{c_1\} \text{ else } \{c_2\}) &\triangleq \text{wf}_C^{\Delta F}(c_1) \wedge \text{wf}_C^{\Delta F}(c_2) \\
\text{wf}_C^{\Delta F}(\text{while}(b) \text{ inv } I \{c\}) &\triangleq \text{wf}_C^{\Delta F}(c) \wedge \text{selfFraming}(I) \wedge \text{stableUnder}(b, I) \wedge \text{wf}_A^{\Delta F}(I) \\
\text{wf}_C^{\Delta F}(\{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}) &\triangleq \text{wf}_C^{\Delta F}(c_1) \wedge \text{wf}_C^{\Delta F}(c_2) \wedge \\
&\quad \text{selfFraming}(P_1) \wedge \text{selfFraming}(Q_1) \wedge \\
&\quad \text{selfFraming}(P_2) \wedge \text{selfFraming}(Q_2) \wedge \\
&\quad \text{wr}(c_1) \cap (\text{fv}_C(c_2) \cup \text{fv}_A(P_2) \cup \text{fv}_A(Q_2)) = \emptyset \wedge \\
&\quad \text{wr}(c_2) \cap (\text{fv}_C(c_1) \cup \text{fv}_A(P_1) \cup \text{fv}_A(Q_1)) = \emptyset \wedge \\
&\quad \text{wf}_A^{\Delta F}(P_1) \wedge \text{wf}_A^{\Delta F}(Q_1) \wedge \text{wf}_A^{\Delta F}(P_2) \wedge \text{wf}_A^{\Delta F}(Q_2) \\
\text{fv}_C(\text{skip}) &\triangleq \emptyset \\
\text{fv}_C(c_1; c_2) &\triangleq \text{fv}_C(c_1) \cup \text{fv}_C(c_2)
\end{aligned}$$

$$\begin{aligned}
\text{fv}_C(\text{if}(b) \{c_1\} \text{ else } \{c_2\}) &\triangleq \text{fv}_B(b) \cup \text{fv}_C(c_1) \cup \text{fv}_C(c_2) \\
\text{fv}_C(\text{while}(b) \text{ inv } I \{c\}) &\triangleq \text{fv}_B(b) \cup \text{fv}_C(c) \\
\text{fv}_C(\{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}) &\triangleq \text{fv}_C(c_1) \cup \text{fv}_C(c_2) \\
\text{wr}(\text{skip}) &\triangleq \emptyset \\
\text{wr}(c_1; c_2) &\triangleq \text{wr}(c_1) \cup \text{wr}(c_2) \\
\text{wr}(\text{if}(b) \{c_1\} \text{ else } \{c_2\}) &\triangleq \text{wr}(c_1) \cup \text{wr}(c_2) \\
\text{wr}(\text{while}(b) \text{ inv } I \{c\}) &\triangleq \text{wr}(c) \\
\text{wr}(\{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}) &\triangleq \text{wr}(c_1) \cup \text{wr}(c_2)
\end{aligned}$$

The small-step operational semantic reduction rules and CSL rules for all the control structures are shown in Fig. 12 and Fig. 13, respectively.

$$\begin{array}{c}
\text{REDPRIM} \quad \frac{\sigma \xrightarrow{\Delta_F} \sigma'}{\langle c, \sigma \rangle \rightarrow_{\Delta_F} \langle \text{skip}, \sigma' \rangle} \qquad \text{REDSEQSKIP} \quad \frac{}{\langle \text{skip}; c, \sigma \rangle \rightarrow_{\Delta_F} \langle c, \sigma \rangle} \qquad \text{REDSEQ} \quad \frac{\langle c_1, \sigma \rangle \rightarrow_{\Delta_F} \langle c'_1, \sigma' \rangle}{\langle c_1; c_2, \sigma \rangle \rightarrow_{\Delta_F} \langle c'_1; c_2, \sigma' \rangle} \\
\\
\text{REDIFT} \quad \frac{I_B(b) (\text{embed}_S(\sigma)) = \text{true}}{\langle \text{if}(b) \{c_1\} \text{ else } \{c_2\}, \sigma \rangle \rightarrow_{\Delta_F} \langle c_1, \sigma \rangle} \qquad \text{REDIFF} \quad \frac{I_B(b) (\text{embed}_S(\sigma)) = \text{false}}{\langle \text{if}(b) \{c_1\} \text{ else } \{c_2\}, \sigma \rangle \rightarrow_{\Delta_F} \langle c_2, \sigma \rangle} \\
\\
\text{REDLOOP} \quad \langle \text{while}(b) \text{ inv } I \{c\}, \sigma \rangle \rightarrow_{\Delta_F} \langle \text{if}(b) \{c; \text{while}(b) \text{ inv } I \{c\} \} \text{ else } \{ \text{skip} \}, \sigma \rangle \\
\\
\text{REDPARSKIP} \quad \langle \{P_1\} \text{ skip } \{Q_1\} \parallel \{P_2\} \text{ skip } \{Q_2\}, \sigma \rangle \rightarrow_{\Delta_F} \langle \text{skip}, \sigma \rangle \\
\\
\text{REDPARL} \quad \frac{\langle c_1, \sigma \rangle \rightarrow_{\Delta_F} \langle c'_1, \sigma' \rangle}{\langle \{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}, \sigma \rangle \rightarrow_{\Delta_F} \langle \{P_1\} c'_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}, \sigma' \rangle} \\
\\
\text{REDPARR} \quad \frac{\langle c_2, \sigma \rangle \rightarrow_{\Delta_F} \langle c'_2, \sigma' \rangle}{\langle \{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\}, \sigma \rangle \rightarrow_{\Delta_F} \langle \{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c'_2 \{Q_2\}, \sigma' \rangle}
\end{array}$$

Fig. 12. Small-step operational semantic reduction rules of the form $\langle c, \sigma \rangle \rightarrow_{\Delta_F} \langle c', \sigma' \rangle$ for front-end statements, where c and c' are statements, σ and σ' are program states, Δ_F is a front-end type context.

The full definition of $\llbracket \cdot \rrbracket_A$ is shown in Fig. 14.

The full definition of $\llbracket \cdot \rrbracket_C$ is as follows, where $C_p \in C_{Fp}$ is a front-end primitive statement, and **havoc** V is a shorthand for **havoc** $x_1; \dots; \text{havoc } x_n$ for finite set $V = \{x_1, \dots, x_n\}$. Here we assume that the IVL supports (1) disjunction and negation of pure boolean expressions, and (2) directly using pure boolean expressions as assertions. It can therefore construct assertions including $\llbracket I \rrbracket_A * (\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B)$ as in the translation of front-end loops.

$$\begin{aligned}
\llbracket \text{skip} \rrbracket_C &\triangleq (\text{skip}, \emptyset) \\
\llbracket C_p \rrbracket_C &\triangleq (\llbracket C_p \rrbracket_{C_p}, \emptyset) \\
\llbracket c_1; c_2 \rrbracket_C &\triangleq (\llbracket c_1 \rrbracket_C^{\text{main}}; \llbracket c_2 \rrbracket_C^{\text{main}}, \llbracket c_1 \rrbracket_C^{\text{ext}} \cup \llbracket c_2 \rrbracket_C^{\text{ext}}) \\
\llbracket \text{if}(b) \{c_1\} \text{ else } \{c_2\} \rrbracket_C &\triangleq (\text{if}(\llbracket b \rrbracket_B) \{ \llbracket c_1 \rrbracket_C^{\text{main}} \} \text{ else } \{ \llbracket c_2 \rrbracket_C^{\text{main}} \}, \llbracket c_1 \rrbracket_C^{\text{ext}} \cup \llbracket c_2 \rrbracket_C^{\text{ext}}) \\
\llbracket \text{while}(b) \text{ inv } I \{c\} \rrbracket_C &\triangleq (\text{exhale } \llbracket I \rrbracket_A * (\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B); \text{havoc } \text{wr}(c); \text{inhale } \llbracket I \rrbracket_A * \neg \llbracket b \rrbracket_B,
\end{aligned}$$

$\frac{\text{RULESKIP} \quad \text{selfFraming}(P)}{\Delta_F \vdash_{\text{CSL}} \{P\} \text{ skip } \{P\}}$	$\frac{\text{RULEPRIM} \quad \text{selfFraming}(P) \quad \text{selfFraming}(Q) \quad \Delta_F \vdash_P \{P\} c \{Q\}}{\Delta_F \vdash_{\text{CSL}} \{P\} c \{Q\}}$
$\frac{\text{RULESEQ} \quad \Delta_F \vdash_{\text{CSL}} \{P\} c_1 \{R\} \quad \Delta_F \vdash_{\text{CSL}} \{R\} c_2 \{Q\}}{\Delta_F \vdash_{\text{CSL}} \{P\} c_1; c_2 \{Q\}}$	
$\frac{\text{RULEIF} \quad \text{frames}(A, b) \quad \Delta_F \vdash_{\text{CSL}} \{A \cap b\} c_1 \{B_1\} \quad \Delta_F \vdash_{\text{CSL}} \{A \cap \neg b\} c_2 \{B_2\}}{\Delta_F \vdash_{\text{CSL}} \{A\} \text{ if } (b) \{c_1\} \text{ else } \{c_2\} \{B_1 \cup B_2\}}$	
$\frac{\text{RULEWHILE} \quad \text{frames}(P, b) \quad \text{stableUnder}(b, P) \quad \Delta_F \vdash_{\text{CSL}} \{P \cap b\} c \{P\}}{\Delta_F \vdash_{\text{CSL}} \{P\} \text{ while } (b) \text{ inv } I \{c\} \{P \cap \neg b\}}$	
$\frac{\text{RULEPAR} \quad \Delta_F \vdash_{\text{CSL}} \{P_1\} c_1 \{Q_1\} \quad \Delta_F \vdash_{\text{CSL}} \{P_2\} c_2 \{Q_2\} \quad \text{fv}_C(c_1) \cap \text{wr}(c_2) = \emptyset \quad \text{fv}_C(c_2) \cap \text{wr}(c_1) = \emptyset \quad \forall x \in \text{wr}(c_1). P_1 \text{ and } Q_1 \text{ are independent of } x \quad \forall x \in \text{wr}(c_2). P_2 \text{ and } Q_2 \text{ are independent of } x}{\Delta_F \vdash_{\text{CSL}} \{P_1 * P_2\} \{A_1\} c_1 \{B_1\} \parallel \{A_2\} c_2 \{B_2\} \{Q_1 * Q_2\}}$	
$\frac{\text{RULEFRAME} \quad \Delta_F \vdash_{\text{CSL}} \{P\} c \{Q\} \quad \text{selfFraming}(R) \quad \forall x \in \text{wr}(c). R \text{ is independent of } x}{\Delta_F \vdash_{\text{CSL}} \{P * R\} c \{Q * R\}}$	
$\frac{\text{RULECONSEQ} \quad \Delta_F \vdash_{\text{CSL}} \{P\} c \{Q\} \quad P' \subseteq P \quad Q \subseteq Q' \quad \text{selfFraming}(P') \quad \text{selfFraming}(Q')}{\Delta_F \vdash_{\text{CSL}} \{P'\} c \{Q'\}}$	

Fig. 13. CSL rules of the form $\Delta_F \vdash_{\text{CSL}} \{P\} c \{Q\}$ for front-end statements, where Δ_F is a front-end type context, c is a statement, P and Q are semantic assertions (*i.e.*, sets of front-end ghost states). The separating conjunction $P * Q$ of semantic assertions P and Q is defined as $\{\omega \mid \exists \omega_1 \in P, \omega_2 \in Q. \omega = \omega_1 \oplus \omega_2\}$. A semantic assertion P is independent of a local variable x iff: $\forall \omega, v. \omega \in P \Leftrightarrow \omega[v/x] \in P$.

$$\begin{array}{ll}
\llbracket A_P \rrbracket_A \triangleq \llbracket A_P \rrbracket_{A_P} & \llbracket b \rrbracket_A \triangleq \llbracket b \rrbracket_B \\
\llbracket A_1 * A_2 \rrbracket_A \triangleq \llbracket A_1 \rrbracket_A * \llbracket A_2 \rrbracket_A & \llbracket A_1 \vee A_2 \rrbracket_A \triangleq \llbracket A_1 \rrbracket_A \vee \llbracket A_2 \rrbracket_A \\
\llbracket b \Rightarrow A \rrbracket_A \triangleq \llbracket b \rrbracket_B \Rightarrow \llbracket A \rrbracket_A & \llbracket \exists x : t. A \rrbracket_A \triangleq \exists x : \llbracket t \rrbracket_T. \text{typeInv}(t, x) * \llbracket A \rrbracket_A \\
\llbracket P \rrbracket_A \triangleq P & \llbracket \mu P. A \rrbracket_A \triangleq \mu P. \llbracket A \rrbracket_A
\end{array}$$

Fig. 14. Extending the input assertion translation $\llbracket \cdot \rrbracket_{A_P}$ to a translation $\llbracket \cdot \rrbracket_A$ of all front-end assertions.

$$\begin{aligned}
& \left\{ \text{inhale } \llbracket I \rrbracket_A * \llbracket b \rrbracket_A ; \llbracket c \rrbracket_C^{\text{main}} ; \text{exhale } \llbracket I \rrbracket_A * (\llbracket b \rrbracket_B \vee \neg \llbracket b \rrbracket_B) \right\} \cup \llbracket c \rrbracket_C^{\text{ext}} \\
\llbracket \{P_1\} c_1 \{Q_1\} \parallel \{P_2\} c_2 \{Q_2\} \rrbracket_C \triangleq & \left(\text{exhale } \llbracket P_1 * P_2 \rrbracket_A ; \text{havoc } \text{wr}(c_1) \cup \text{wr}(c_2) ; \text{inhale } \llbracket Q_1 * Q_2 \rrbracket_A , \right. \\
& \left. \left\{ \text{inhale } \llbracket P_1 \rrbracket_A ; \llbracket c_1 \rrbracket_C^{\text{main}} ; \text{exhale } \llbracket Q_1 \rrbracket_A \right\} \cup \right. \\
& \left. \left\{ \text{inhale } \llbracket P_2 \rrbracket_A ; \llbracket c_2 \rrbracket_C^{\text{main}} ; \text{exhale } \llbracket Q_2 \rrbracket_A \right\} \cup \llbracket c_1 \rrbracket_C^{\text{ext}} \cup \llbracket c_2 \rrbracket_C^{\text{ext}} \right)
\end{aligned}$$

The requirements for the input front-end components are listed in [Req. 3](#).

Requirement 3. *The following properties need to be satisfied by the input front-end components:*

- I_B is pure, i.e., $\forall b \in BE_F, \omega \in \Sigma_F, I_B(b)(\omega)$ is defined iff $I_B(b)(|\omega|)$ is defined, and $I_B(b)(\omega) = I_B(b)(|\omega|)$ when they are both defined.
- For any boolean expression $b \in BE_F$, $fv_B(b)$ overapproximates the free variables of b , i.e., $\forall x \notin fv_B(b), v \in V_F, \omega \in \Sigma_F, I_B(b)(\omega)$ is defined iff $I_B(b)(\omega[v/x])$ is defined, and $I_B(b)(\omega) = I_B(b)(\omega[v/x])$ when they are both defined.
- For any input assertion $A_p \in A_{Fp}$, $fv_A(A_p)$ overapproximates the free variables of A_p , i.e., $\forall x \notin fv_A(A_p), \omega \in \Sigma_F, v \in V_F, \omega \in I_A^{\Delta_F}(A_p)$ iff $\omega[v/x] \in I_A^{\Delta_F}(A_p)$.
- The written variables of well-formed primitive statements must be declared in the type context, i.e., for any $c \in C_{Fp}$, if $wf_C^{\Delta_F}(c)$, then $wr(c) \subseteq \text{dom}(\Delta_F)$.
- The CSL rules for any well-formed primitive statement C are affine, i.e., if $\Delta_F \vdash_{\text{CSL}} \{P\} C \{Q\}$ for some semantic assertions P and Q , then $\Delta_F \vdash_{\text{CSL}} \{\text{monotonize}(P)\} C \{\text{monotonize}(Q)\}$. Here, for any semantic assertion A , $\text{monotonize}(A) \triangleq \{\omega \mid \exists \omega_0. \omega \succeq \omega_0 \wedge \omega_0 \in A\}$ is the minimal semantic assertion that is monotone (a semantic assertion P is monotone iff $\forall \omega \in P$ and $\omega' \succeq \omega, \omega' \in P$) and that subsumes A .
- The operational semantic reduction rules of any primitive statement $c \in C_{Fp}$ do not modify local variables outside $wr(c)$, i.e., if $\sigma \xrightarrow{c}_{\Delta_F} \sigma'$, then $\forall x \notin wr(c), \sigma(x) = \sigma'(x)$.
- For any primitive statement $c \in C_{Fp}$, any finite number of changes outside $fv_C(c)$ does not affect the reduction of c , i.e., if two states σ_1, σ_2 agree on a set X whose complement is finite and that subsumes $fv_C(c)$, and they have the same program heap, and $\sigma_1 \xrightarrow{c}_{\Delta_F} \sigma'_1$, then there exists σ'_2 such that $\sigma_2 \xrightarrow{c}_{\Delta_F} \sigma'_2$, and σ'_1 and σ'_2 agree on X and the whole program heap.
- The operational semantic reduction of any well-formed primitive statement preserves well-typedness, i.e., if $\sigma \xrightarrow{c}_{\Delta_F} \sigma'$ for some σ that is well-typed under Δ_F , then σ' is also well-typed under Δ_F .
- The precondition of each CSL rule for well-formed primitive statements guarantees progression for well-typed states. Specifically, if $\Delta_F \vdash_p \{P\} c \{Q\}$ for some primitive statement $c \in C_{Fp}$ and monotone semantic assertion $P \subseteq \mathcal{P}(\Sigma_F)$, then for any program state $\sigma_p \in \Sigma_P$ that is well-typed w.r.t. Δ_F , if $\text{embed}_S(\sigma_p) = \sigma \oplus \sigma_f$ such that $\sigma \in P$ and both σ and σ_f are stable, then there exists $\sigma'_p \in \Sigma_P$ such that $\sigma_p \xrightarrow{c}_{\Delta_F} \sigma'_p$.
- The front-end CSL rules for well-formed primitive statements are sound w.r.t. operational semantics: for any well-formed primitive statement, the post-state after the reduction satisfies the postcondition, whenever starting from a well-typed pre-state satisfying the precondition. Specifically, let $c \in C_{Fp}$ be well-formed and σ_p be well-typed w.r.t. Δ_F . If $\Delta_F \vdash_p \{P\} c \{Q\}$ for monotone semantic assertions P and Q , $\text{embed}_S(\sigma_p) = \sigma \oplus \sigma_f$ such that $\sigma \in P$ and both σ and σ_f are stable, and $\sigma_p \xrightarrow{c}_{\Delta_F} \sigma'_p$, then there exist σ' and σ'_f such that σ_f and σ'_f agree on the ghost heap, σ' is stable, $\text{embed}_S(\sigma'_p) = \sigma' \oplus \sigma'_f$, and $\sigma' \in Q$.

The requirements for the translation of front-end types and values are listed in [Req. 4](#).

Requirement 4. For the translation of front-end types and values, $\llbracket \cdot \rrbracket_T$, $\llbracket \cdot \rrbracket_V$, and typeInv should satisfy the following properties:

- $\llbracket \cdot \rrbracket_T$ translates types correctly. Specifically, if $\text{tp}(v) = t$ for any front-end value v and type t , then $\llbracket v \rrbracket_V$ is of type $\llbracket t \rrbracket_T$.
- $\llbracket \cdot \rrbracket_V$ is injective. Specifically, $\llbracket v_1 \rrbracket_V = \llbracket v_2 \rrbracket_V$ implies $v_1 = v_2$ for any front-end values v_1 and v_2 .
- typeInv correctly reflects the bijectivity of the translation of values of all types. Specifically, if $\text{typeInv}(t, x)$ is undefined for front-end type $t \in T_F$ for all x , then any value v of type $\llbracket t \rrbracket_T$ in the IVL must correspond to some front-end value v_f of type t (i.e., $\llbracket v_f \rrbracket_V = v$); if $\text{typeInv}(t, x) = e$,

then for any IVL state $\Omega \in \Sigma_V$ with $\Omega(x) = v$ for v of type $\llbracket t \rrbracket_T$, $e(\Omega) = \text{true}$ iff there exists v_f of type t such that $\llbracket v_f \rrbracket_V = v$.

- The expressions `typeInv` returns are pure and monotone. Specifically, if $\text{typeInv}(t, x) = e$, then for any $\Omega \in \Sigma_V$, $e(\Omega)$ is defined iff $e(|\Omega|)$ is defined, and $e(\Omega) = e(|\Omega|)$ when they are both defined; for any $\Omega, \Omega' \in \Sigma_V$, if $e(\Omega)$ is defined and $\Omega' \succeq \Omega$, then $e(\Omega')$ is also defined and evaluates to the same value as $e(\Omega)$ does.

The requirement for the translation of primitive statements $\llbracket \cdot \rrbracket_{C_p}$ is shown as [Req. 5](#).

Requirement 5. $\llbracket \cdot \rrbracket_{C_p}$ is sound: it preserves the semantics of any primitive statement C . Specifically, if $\llbracket \Delta_F \rrbracket_\Delta \vdash [P] \llbracket C \rrbracket_{C_p} [Q]$ in the IVL for monotone IVL assertions P and Q , then $\Delta_F \vdash_{\text{CSL}} \{ \text{btr}(P) \} C \{ \text{btr}(Q) \}$ in the CSL of the front-end. Here, for any IVL semantic assertion A , $\text{btr}(A) \triangleq \{ \omega \mid \llbracket \omega \rrbracket_S \in A \}$ is the backward translation of A , i.e., a collection of all front-end states whose encoded states satisfy A .

We prove [Thm. 7](#) in the framework, which states the soundness of the CSL rules in [Fig. 13](#) w.r.t. the small-step operational semantics in [Fig. 12](#).

Theorem 7 (Soundness of the CSL in [Fig. 13](#)). If $\Delta_F \vdash_{\text{CSL}} \{P\} C \{Q\}$ for well-formed front-end statement C and monotone P and Q , then for any well-typed stable program state σ such that $\text{embed}_S(\sigma) \in P$, executing C from σ never aborts, and for all σ' that $\langle C, \sigma \rangle \rightarrow_{\Delta_F}^* \langle \text{skip}, \sigma' \rangle$, we have $\text{embed}_S(\sigma') \in Q$. Here $\langle \cdot, \cdot \rangle \rightarrow^* \langle \cdot, \cdot \rangle$ is the reflexive transitive closure of $\langle \cdot, \cdot \rangle \rightarrow \langle \cdot, \cdot \rangle$.

B Full Definitions and Semantics of the Instantiated Front-Ends

B.1 Encoding Rational Permission Amounts into Real Permission Amounts

The interpretation of $\text{acc}(x.f, p)$ is defined in [Fig. 15](#).

AccPos

$$\frac{a \text{ is a heap location identifier} \quad \begin{array}{c} I_E(x)((s, (h, \pi))) = a \\ I_E(p) = p_v > 0 \quad \pi(a.f) \geq p_v \quad h(a.f) = v \quad \Delta_F(f) = \text{tp}(v) \end{array}}{(s, (h, \pi)) \in I_A^{\Delta_F}(\text{acc}(x.f, p))}$$

AccZERO

$$\frac{I_E(x)((s, (h, \pi))) = r \quad r = \text{null} \text{ or } r \text{ is a heap location identifier} \quad I_E(p) = 0 \quad \Delta_F(f) \text{ is defined}}{(s, (h, \pi)) \in I_A^{\Delta_F}(\text{acc}(x.f, p))}$$

Fig. 15. The interpretation of $\text{acc}(x.f, p)$ in the rational permissioned front-end.

Every input assertion is well-formed in this front-end, i.e., $\text{wf}_A^{\Delta_F}(A_p) \triangleq \text{true}$ for any input assertion A_p .

We can define syntactically the type $\text{tp}_{\Delta_F}(e)$ for each expression e by an induction on the structure of e . The definition guarantees that: for any expression e of type $\text{tp}_{\Delta_F}(e) = t$, if it evaluates to v at some well-typed state ω , then v is of type t . Using this definition, the well-formedness predicate for primitive statements is defined as follows.

$$\begin{aligned} \text{wf}_C^{\Delta_F}(x := e) &\triangleq \exists t. \Delta_F(x) = t \wedge \text{tp}_{\Delta_F}(e) = t \\ \text{wf}_C^{\Delta_F}(x.f := e) &\triangleq \text{tp}_{\Delta_F}(x) = \text{Ref} \wedge (\exists t. \Delta_F(f) = t \wedge \text{tp}_{\Delta_F}(e) = t) \\ \text{wf}_C^{\Delta_F}(x := \text{alloc}(f_1, \dots, f_n)) &\triangleq \Delta_F(x) = \text{Ref} \wedge f_1, \dots, f_n \text{ are distinct} \wedge \forall i. \Delta_F(f_i) \text{ is defined} \\ \text{wf}_C^{\Delta_F}(\text{free}(x.f)) &\triangleq \top \end{aligned}$$

The operational semantic reduction rules and CSL rules for the primitive statements are listed in Fig. 16 and Fig. 17, respectively.

$$\begin{array}{c}
\text{REDASSIGNVAR} \\
\frac{e(\sigma) = v}{\sigma \xrightarrow{x:=e}_{\Delta_F} \sigma[v/x]} \\
\\
\text{REDASSIGNFIELD} \\
\frac{x(\sigma) = a \quad a \text{ is a heap location identifier} \quad e(\sigma) = v \quad \text{tp}(v) = \Delta_F(f) \quad \sigma(a.f) \neq \perp}{\sigma \xrightarrow{x.f:=e}_{\Delta_F} \sigma[a.f := v]} \\
\\
\text{REDALLOC} \\
\frac{\forall f. (l.f \notin \text{dom}(\sigma)) \quad \Delta_F(x) = \text{Ref} \quad \text{length}(\vec{v}) = n \quad \forall 0 < i \leq n. \text{tp}(\vec{v}[i]) = \Delta_F(f_i)}{\sigma \xrightarrow{x:=\text{alloc}(f_1, \dots, f_n)}_{\Delta_F} \sigma[l/x][l.f_1 := \vec{v}[1], \dots, l.f_n := \vec{v}[n]]}
\end{array}$$

Fig. 16. Operational semantic reduction rules for primitive statements of the ractional permissioned front-end. We use $e(\sigma)$ to represent the evaluation of e on program state σ (whose result is the same as $I_E(e)$ ($\text{embed}_S(\sigma)$)), $\sigma[v/x]$ to represent the state by substituting the value of local variable x by value v in σ , and $\sigma[a.f := \perp]$ or $\sigma[a.f := v]$ to represent the state by erasing the value at location $a.f$ or setting the value to v in σ , respectively.

$$\begin{array}{c}
\text{RULEASSIGNVAR} \\
\frac{\text{selfFraming}(A) \quad \text{frames}(A, e)}{\Delta_F \vdash_P \{A\} x := e \{A[e/x]\}} \\
\\
\text{RULEASSIGNFIELD} \\
\frac{\text{selfFraming}(A) \quad A \Rightarrow x.f \mapsto _ \quad \text{frames}(A, e)}{\Delta_F \vdash_P \{A\} x.f := e \{A[x.f := e]\}} \\
\\
\text{RULEALLOC} \\
\Delta_F \vdash_P \{\top\} x := \text{alloc}(f_1, \dots, f_n) \{\text{acc}(x.f_1, 1) * \dots * \text{acc}(x.f_n, 1)\} \\
\\
\text{RULEFREE} \\
\Delta_F \vdash_P \{x.f \mapsto _ \} \text{free}(x.f) \{\top\}
\end{array}$$

Fig. 17. CSL rules for primitive statements of the ractional permissioned front-end. Here $\text{acc}(x.f, 1)$ represents the typed version of $x.f \mapsto _$, i.e., while $x.f \mapsto _$ asserts that the current state has full permission to the heap location $x.f$, $\text{acc}(x.f, 1)$ further requires that the value $\omega(x.f)$ is well-typed w.r.t. the type context Δ_F . We use $A[e/x]$ for the assertion $\{\omega[I_E(e)(\omega)/x] \mid \omega \in A\}$, i.e., updating the value of the local variable x to the evaluation of e for every state in A , and $A[x.f := e]$ for the assertion $\{\omega[a.f := v] \mid \omega \in A, I_E(x)(\omega) = a \text{ for some heap location identifier } a, I_E(e)(\omega) = v\}$, i.e., updating the value of the heap location $x(\omega).f$ to $e(\omega)$ for every $\omega \in A$ that x and e evaluate correctly.

The translation of all four primitive statements are listed as follows.

$$\begin{aligned}
\llbracket x := e \rrbracket_{\text{Cp}} &\triangleq x := e \\
\llbracket x.f := e \rrbracket_{\text{Cp}} &\triangleq x.f := e \\
\llbracket x := \text{alloc}(f_1, \dots, f_n) \rrbracket_{\text{Cp}} &\triangleq \text{havoc } x; \text{inhale } \text{acc}(x.f_1, 1) * \dots * \text{acc}(x.f_n, 1) \\
\llbracket \text{free}(x.f) \rrbracket_{\text{Cp}} &\triangleq \text{exhale } \text{acc}(x.f, 1)
\end{aligned}$$

A sound fragment of magic wands. The interpretation of $L \multimap R$ is defined as follows:

$$I_A^{\Delta_F}(L \multimap R) \triangleq \left\{ \omega \mid \forall \omega_L \in I_A^{\Delta_F}(L). \omega_L \text{ is well-typed w.r.t. } \Delta_F \text{ and compatible with } \omega \Rightarrow \omega_L \oplus \omega \in I_A^{\Delta_F}(R) \right\}$$

Well-formedness of $L \multimap R$ is defined as $\text{wf}_A^{\Delta_F}(L \multimap R) \triangleq \text{noRatAcc}^{\Delta_F}(L) \wedge \text{noRatAcc}^{\Delta_F}(R)$, where $\text{noRatAcc}^{\Delta_F}(A_b)$ if and only if A_b does not access rational fields. Its concrete definition is given in

Fig. 18, where $\text{noRatAcc}_E^{\Delta_F}(e)$ of an expression e holds if and only if the type of any field name f in e is not **Rat** in Δ_F .

$$\begin{aligned}
\text{noRatAcc}^{\Delta_F}(\text{acc}(x.f, 1)) &\triangleq \text{noRatAcc}_E^{\Delta_F}(x) \\
\text{noRatAcc}^{\Delta_F}(e) &\triangleq \text{noRatAcc}_E^{\Delta_F}(e) \\
\text{noRatAcc}^{\Delta_F}(A_1 * A_2) &\triangleq \text{noRatAcc}^{\Delta_F}(A_1) \wedge \text{noRatAcc}^{\Delta_F}(A_2) \\
\text{noRatAcc}^{\Delta_F}(A_1 \vee A_2) &\triangleq \text{noRatAcc}^{\Delta_F}(A_1) \wedge \text{noRatAcc}^{\Delta_F}(A_2) \\
\text{noRatAcc}^{\Delta_F}(e \Rightarrow A) &\triangleq \text{noRatAcc}_E^{\Delta_F}(e) \wedge \text{noRatAcc}^{\Delta_F}(A) \\
\text{noRatAcc}^{\Delta_F}(\exists x : t. A) &\triangleq \text{noRatAcc}^{\Delta_F}(A)
\end{aligned}$$

Fig. 18. Definition of $\text{noRatAcc}^{\Delta_F}$.

B.2 Encoding Lazy Field Creation

$I_A^{\Delta_F}$ for the three input assertions is shown in Fig. 19. Every input assertion is well-formed in this front-end, i.e., $w_A^{\Delta_F}(A_p) \triangleq \text{true}$ for any input assertion A_p .

$$\begin{array}{c}
\text{DFCAcc} \\
\frac{I_E(x)((s, (h, \pi))) = a \quad a \text{ is a heap location identifier} \quad \pi(a.f) \geq p_v > 0 \quad h(a.f) = \text{nofield} \vee (h(a.f) = v \wedge \Delta_F(f) = \text{tp}(v))}{I_A^{\Delta_F}(\text{acc}(x.f, p)) = 1} \\
\\
\text{DFCAdded} \\
\frac{I_E(x)((s, (h, \pi))) = a \quad a \text{ is a heap location identifier} \quad h(a.f) = v \neq \text{nofield}}{I_A^{\Delta_F}(\text{added}(x.f)) = 1} \\
\\
\text{DFCMissing} \\
\frac{I_E(x)((s, (h, \pi))) = a \quad a \text{ is a heap location identifier} \quad h(a.f) = \text{nofield}}{I_A^{\Delta_F}(\text{missing}(x.f)) = 1}
\end{array}$$

Fig. 19. The assertion interpretation $I_A^{\Delta_F}$ for input assertions of the lazy field creation front-end.

The operational semantic reduction rule for allocations is shown in Fig. 20.

$$\begin{array}{c}
\text{REDALLOC} \\
\frac{\forall f.(l, f) \notin \text{dom}(\omega) \quad \Delta_F(x) = \text{Ref}}{\omega \xrightarrow{x := \text{alloc}(f_1, \dots, f_n)}_{\Delta_F} \omega[l/x][l.f_i := \text{nofield}]}
\end{array}$$

Fig. 20. The operational semantic reduction rule for $x := \text{alloc}(f_1, \dots, f_n)$ in the lazy field creation front-end.

The translation of input assertions is defined as follows:

$$\begin{aligned}
\llbracket \text{acc}(x.f, p) \rrbracket_{\text{Ap}} &\triangleq \llbracket p \rrbracket_E > 0 * \text{acc}(\llbracket x \rrbracket_E.f_{\text{added}}, \llbracket p \rrbracket_E) * (\llbracket x \rrbracket_E.f_{\text{added}} = \text{true} \Rightarrow \text{acc}(\llbracket x \rrbracket_E.f_{\text{IVL}}, \llbracket p \rrbracket_E)) \\
\llbracket \text{added}(x.f) \rrbracket_{\text{Ap}} &\triangleq \llbracket x \rrbracket_E.f_{\text{added}} = \text{true} \\
\llbracket \text{missing}(x.f) \rrbracket_{\text{Ap}} &\triangleq \llbracket x \rrbracket_E.f_{\text{added}} = \text{false}
\end{aligned}$$

The translation of primitive statements is defined as follows.

$$\begin{aligned}
\llbracket x := e \rrbracket_{\text{Cp}} &\triangleq x := \llbracket e \rrbracket_{\text{E}} \\
\llbracket x := \text{alloc}(f_1, \dots, f_n) \rrbracket_{\text{Cp}} &\triangleq \text{havoc } x; \text{inhale } \llbracket \text{acc}(x.f_1, 1) \rrbracket_{\text{A}} * \dots * \llbracket \text{acc}(x.f_n, 1) \rrbracket_{\text{A}} \\
\llbracket \text{free}(x.f) \rrbracket_{\text{Cp}} &\triangleq \text{exhale } \llbracket \text{acc}(x.f, 1) \rrbracket_{\text{A}} \\
\llbracket x.f := v \rrbracket_{\text{Cp}} &\triangleq \text{if}(\llbracket x \rrbracket_{\text{E}}.f_{\text{added}} = \text{false}) \{ \text{inhale } \text{acc}(\llbracket x \rrbracket_{\text{E}}.f_{\text{IVL}}, 1) \} \text{ else } \{ \text{skip} \}; \\
&\quad \llbracket x \rrbracket_{\text{E}}.f_{\text{added}} := \text{true}; \llbracket x \rrbracket_{\text{E}}.f_{\text{IVL}} := v
\end{aligned}$$

B.3 Encoding Immutable Heaps

The interpretation function $I_A^{\Delta_F}$ on input assertion $\text{init}(x_{\circ}f)$ is shown in Fig. 21. Every input assertion is well-formed in this front-end, i.e., $\text{wf}_A^{\Delta_F}(A_p) \triangleq \text{true}$ for any input assertion A_p .

$$\frac{\text{IMINIT} \quad I_E(x)(s, (h, \pi, h_{\text{im}})) = a \quad a \text{ is a heap location identifier} \quad h_{\text{im}}(a_{\circ}f) = v \quad \text{tp}(v) = \Delta_F^{\text{im}}(f)}{(s, (h, \pi, h_{\text{im}})) \in I_A^{\Delta_F}(\text{init}(x_{\circ}f))}$$

Fig. 21. The interpretation of $\text{init}(x_{\circ}f)$ in the immutable heap front-end. We allow the regular heap and the immutable heap to have different type contexts, and use Δ_F^{im} to denote the type context of the immutable heap.

The free variable set, written variable set, and well-formedness predicate for $x := \text{imalloc}(f := v)$ are naturally defined as follows.

$$\begin{aligned}
\text{fv}_C(x := \text{imalloc}(f := v)) &\triangleq \{x, v\} \\
\text{wr}(x := \text{imalloc}(f := v)) &\triangleq \{x\} \\
\text{wf}_C^{\Delta_F}(x := \text{imalloc}(f := v)) &\triangleq x \neq v \wedge \Delta_F(x) = \text{Ref} \wedge \Delta_F(v) = \Delta_F(f) \neq \perp
\end{aligned}$$

The CSL rule and operational semantic reduction rule for $x := \text{imalloc}(f := v)$ are shown in Fig. 22.

$$\begin{array}{c}
\text{RULEINIT} \\
\Delta_F \vdash_{\text{CSL}} \{ \top \} x := \text{imalloc}(f := v) \{ x_{\circ}f = v \}
\end{array}
\quad
\frac{\text{REDINIT} \quad \forall f. \omega(l_{\circ}f) = \perp \quad \Delta_F(x) = \text{Ref} \quad \omega(v) = v_0}{\omega \xrightarrow{x := \text{imalloc}(f := v)}_{\Delta_F} \omega[l/x][l_{\circ}f := v_0]}$$

Fig. 22. The CSL rule and operational semantic reduction rule for the primitive statement $x := \text{imalloc}(f := v)$ in the immutable heap front-end.

The translation of primitive statements is listed as follows.

$$\begin{aligned}
\llbracket x := e \rrbracket_{\text{Cp}} &\triangleq x := \llbracket e \rrbracket_{\text{E}} \\
\llbracket x.f := e \rrbracket_{\text{Cp}} &\triangleq \llbracket x \rrbracket_{\text{E}}.f_h(f) := \llbracket e \rrbracket_{\text{E}} \\
\llbracket x := \text{alloc}(f_1, \dots, f_n) \rrbracket_{\text{Cp}} &\triangleq \text{havoc } x; \text{inhale } \text{acc}(x.f_h(f_1), 1) * \dots * \text{acc}(x.f_h(f_n), 1) \\
\llbracket \text{free}(x.f) \rrbracket_{\text{Cp}} &\triangleq \text{exhale } \text{acc}(x.f_h(f), 1) \\
\llbracket x := \text{imalloc}(f := v) \rrbracket_{\text{Cp}} &\triangleq \text{havoc } x; \text{inhale } \text{acc}(x.f_v(f), _) * x.f_v(f) = v
\end{aligned}$$